"Granite: A Self-Healing Multi-Agent System for Semantic Error Diagnosis, Verified Repair and Recovery"

Project Team

Shahoud Shahid	23i-2515
Muhammad Noor Ul Haq	23i-2520
Saif Shahzad	23i-2634

Session 2023-2027

Supervised by

Dr. Adil Majeed



FAST School of Computing

National University of Computer and Emerging Sciences
Islamabad, Pakistan

October, 2026

Contents

Students' Submission i

Anti-Plagiarism Statement iii

Authors' Declaration v

Abstract vii

Acknowledgments ix

Executive Summary xi

List of Abbreviations xiii

1 Introduction and Project Vision 1

1.1 Project Overview . . . 1
1.2 Problem Statement . . . 2
1.3 Problem Context and Motivation . . . 2
1.4 Project Vision . . . 3
1.5 Project Objectives . . . 4
1.6 Project Scope . . . 5
1.7 Project Modules . . . 6
1.8 Stakeholders and User Classes . . . 7
1.9 Success Criteria . . . 8
1.10 Constraints and Assumptions . . . 8
1.11 Ethical, Legal and DEI Considerations . . . 9

2 Research and Existing Solutions 11

2.1 Research Method / Approach . . . 11
2.2 Existing Solutions / Products . . . 12
2.3 Comparison of Existing Solutions . . . 13
2.4 Limitations and Gaps . . . 15
2.5 Proposed Solution . . . 15
2.6 Complex Computing Problem . . . 16

2.7 Technical Challenges 16
2.8 Research Findings and Design Decisions 17

3 Proposal Feedback and Iteration Plan **19**
3.1 Proposal Feedback Summary 19
3.2 Proposal Action Register 20
3.3 Iteration Strategy 21
3.4 Iteration 1 Plan 22
3.5 Iteration 2 Plan 23
3.6 Work Breakdown and Responsibilities 23
3.7 Progress Against Plan 24
3.8 Delays, Deviations and Recovery Plan 24

4 Software Requirements **27**
4.1 User Interactions / Use Cases 27
4.2 Use Case Diagram 29
4.3 Detailed Use Cases 29
4.4 Functional Requirements 32
4.4.1 Module 1: Run 33
4.4.2 Module 2: Check 34
4.4.3 Module 3: Trace back 35
4.4.4 Module 4: Verify 35
4.4.5 Module 5: Review, Report and History 36
4.5 Non-Functional Requirements 36
4.5.1 Reliability 37
4.5.2 Usability 37
4.5.3 Performance 37
4.5.4 Security 37
4.6 Acceptance Criteria 38
4.7 Requirements Traceability 38

5 System Design **41**
5.1 System Architecture 41
5.2 Architectural Components 42
5.3 Interfaces and Dependencies 43
5.4 Architecture Decisions and Rationale 45
5.5 Design Evolution 46
5.6 Domain Model 47
5.7 Domain Model 47
5.8 Data Design 47
5.9 Process / Behavior Design 48

6 Iteration 1: Design and Implementation **53**
6.1 Iteration Objectives . . . 53
6.2 Selected Use Cases / Features . . . 53
6.3 Design . . . 54
6.4 Implementation . . . 55
6.5 Completed Features / Module . . . 55
6.6 Technical Implementation Details . . . 56
6.7 Algorithms and Technical Decisions . . . 57
6.8 Tools, Frameworks and Technologies . . . 60
6.9 Demonstration Evidence . . . 60
6.10 Iteration Outcome . . . 65

7 Testing and Early Validation **67**
7.1 Testing Strategy . . . 67
7.2 Test Levels and Techniques . . . 68
7.3 Test Cases . . . 69
7.4 Test Results . . . 70
7.5 Acceptance Testing / Validation . . . 70
7.6 Edge and Failure Cases . . . 71
7.7 Defects and Corrective Actions . . . 72
7.8 Current Validation Status . . . 73

8 Project Management and Development Evidence **75**
8.1 Work Completed . . . 75
8.2 Work Remaining . . . 77
8.3 Individual Responsibilities . . . 78
8.4 Contribution of Each Team Member . . . 79
8.5 Repository and Version Control Evidence . . . 80
8.6 Milestones and Commit Evidence . . . 80
8.7 Tools and Tool-Assisted Development . . . 81
8.8 External Components and Dependencies . . . 82
8.9 Risks and Mitigation . . . 82
8.10 Security and Privacy Considerations . . . 83
8.11 Current Limitations . . . 84

9 Remaining FYP Plan **87**
9.1 Remaining Scope . . . 87
9.2 Remaining Modules and Features . . . 88
9.3 Remaining Iterations . . . 89
9.4 Implementation Plan . . . 89
9.5 Testing and Validation Plan . . . 90

9.6 Deployment Plan 92
9.7 Tentative Timeline 94
9.8 Expected Final Deliverables 94

10 Conclusion **97**

A Proposal Action Register **99**

B Detailed Requirements **103**

C Detailed Test Cases **105**

D Repository and Contribution Evidence **107**

E Additional Design Diagrams **109**

F Meeting and Progress Evidence **113**

List of Figures

1.1 The Refund Desk workflow: five agents, the data each step reads, and the seven checks . . . 3

3.1 Panel comment A1: the proposal-defense PoC (left) compared with the Mid-1 product (right) . . . 22

4.1 Use case diagram (implemented and planned use cases are drawn differently) 29
4.2 System sequence for UC-03, running a workflow . . . 32
4.3 System sequence for UC-08, verifying a hypothesis by controlled replay . 33

5.1 Layer diagram: one dependency direction, checked in CI . . . 42
5.2 Component diagram . . . 43
5.3 Domain model (class diagram) generated from the code . . . 48
5.4 Entity relationship diagram of the twelve tables . . . 49
5.5 Finding state machine, read from the lifecycle code . . . 50
5.6 Activity diagram of an investigation . . . 51
5.7 Data flow diagram, level 0 (context) . . . 51
5.8 Data flow diagram, level 1 . . . 52

6.1 Structure chart with the real function names . . . 56
6.2 Check language: from expression text to result . . . 58
6.3 Attribution: ranking where a failure began . . . 59
6.4 Verdict rule VR-1 . . . 59
6.5 Replay plan: correct one step, keep everything before it . . . 60
6.6 Investigations list: earlier runs and findings, with filters and search . . . 61
6.7 New run: validated input form . . . 62
6.8 Run page: pipeline graph with failed steps and the seven checks . . . 62
6.9 Finding page: ranked hypotheses, verification results, review and evidence 63
6.10 Failure case: a run that cannot be answered ends as errored with the reason 64
6.11 Guide page: the eight features with their inputs and outputs (feature F8) . 64

7.1 Smoke test passing against the public deployment . . . 70

9.1 Deployment diagram: verified and planned parts . . . 93
9.2 The granite-demo service live on Render . . . 93

E.1 Investigation flow diagram 110
E.2 Call graph computed from the source (38 functions, 49 calls) 111
E.3 Interaction overview diagram 111

List of Tables

1.1 Project Scope 6
1.2 Stakeholders and User Classes 7
1.3 Success Criteria 8

2.1 Comparison of Existing Solutions 14
2.2 Complex Computing Problem 16
2.3 Research Findings and Design Decisions 17

3.1 Proposal Action Register 20
3.2 Work Breakdown and Responsibilities 23

4.1 User Interactions / Use Cases 28
4.2 Module 1: Run 33
4.3 Module 2: Check 34
4.4 Module 3: Trace back 35
4.5 Module 4: Verify 35
4.6 Module 5: Review, Report and History 36
4.7 Acceptance Criteria 38
4.8 Requirements Traceability 39

5.1 System Architecture 42
5.2 Interfaces and Dependencies 44
5.3 Architecture Decisions and Rationale 45
5.4 Design Evolution 46

6.1 Implementation 55

7.1 Test Levels and Techniques 68
7.2 Feasibility validation results 71

8.1 Automated test results, 11 October 2026 76
8.2 Responsibility matrix (R responsible, A accountable, C consulted, I informed) 79
8.3 Principal risks 83

9.1 Implementation Plan 89

9.2 Tentative Timeline 94

A.1 Proposal Action Register 100

B.1 Detailed Requirements 103

C.1 Detailed Test Cases 105

E.1 Additional Design Diagrams 109

F.1 Meeting and Progress Evidence 113
F.2 Meeting and Progress Evidence 113

Students' Submission

We certify that this report is submitted as the FYP-1 mid-report for the project titled *"Granite: A Self-Healing Multi-Agent System for Semantic Error Diagnosis, Verified Repair and Recovery"*.

Student 1: Shahoud Shahid (23i-2515) Signature: _____

Student 2: Muhammad Noor Ul Haq (23i-2520) Signature: _____

Student 3: Saif Shahzad (23i-2634) Signature: _____

Anti-Plagiarism Statement

The students certify that this report is their own work. All sources used in preparing it have been properly acknowledged and referenced.

The undersigned members of project F26-060 (Granite) attest to the originality of this report. When we use someone else's ideas, words, data, figures or code, we credit and cite them. They are also listed in the References section.

Specifically, we affirm that:

- All research, products and datasets in this report are cited to their original sources. This includes the papers reviewed in Chapter 2 and the tau-bench retail data used by our workflow.
- Third-party software and its licences are listed in Section 8.8. We use each component under its licence terms.
- We used generative AI and other tools for drafting, coding and analysis. This is disclosed in Section 8.7, our AI usage log and Form 13. The team reviewed, tested or rewrote all tool-assisted work and takes full responsibility for it.
- No part of this report has been submitted for any other course or project.
- All results, including the tests and the feasibility evaluation, are reported as obtained. No data has been made up or changed.

Authors' Declaration

This report covers the work our team has completed up to the FYP-1 mid evaluation.

The system we describe is built and running. Work that is still planned is marked as planned.

Section 8.4 explains what each member contributed. Section 8.7 lists the tools we used, including AI tools. We credit outside components and sources wherever we use them.

To the best of our knowledge, everything in this report is true and complete.

Abstract

We are building Granite, a self-hosted tool that closes this gap. It detects silent failures with automatic checks against trusted data. It follows the wrong value back to the step where it started. It then proves the cause by re-running the workflow with that step fixed and comparing it with unfixed runs. A person makes the final decision, and all evidence is stored.

We work in iterations, each ending in a working product. Iteration 1 is built and deployed: a five-agent refund workflow on public data that passes 381 automated tests. In a test set up in advance on ten tasks, Granite caught every injected fault, found the true cause each time and confirmed it in nine. This shows the approach works end to end.

Iteration 2 adds more fault types, imported traces and fix suggestions. FYP-2 adds a full benchmark, baseline comparisons and user testing. **Multi-agent systems built on large language models often fail silently. The workflow finishes, the answer sounds right, and no error appears, but the result is wrong. Current research finds the faulty step mainly by asking a language model to read the logs, and its accuracy is low. Commercial tools do not prove their diagnoses.**

We are building Granite, a self-hosted tool that closes this gap. It detects silent failures with automatic checks against trusted data. It follows the wrong value back to the step where it started. It then proves the cause by re-running the workflow with that step fixed and comparing it with unfixed runs. A person makes the final decision, and all evidence is stored.

We work in iterations, each ending in a working product. Iteration 1 is built and deployed: a five-agent refund workflow on public data that passes 381 automated tests. In a test set up in advance on ten tasks, Granite caught every injected fault, found the true cause each time and confirmed it in nine. This shows the approach works end to end.

Iteration 2 adds more fault types, imported traces and fix suggestions. FYP-2 adds a full benchmark, baseline comparisons and user testing.

Acknowledgments

We are grateful to our supervisor, Dr. Adil Majeed, for the guidance and for encouraging us to turn a research idea into a working product.

We also thank the members of the proposal defense panel. Their comments on the interface, features, inputs, verification and workflow shaped the redesign in this report.

We thank the FAST School of Computing for the FYP framework and resources.

Our Refund Desk workflow uses the tau-bench retail data, released by Sierra under the MIT licence.

Executive Summary

The problem. When several AI agents work together, an early mistake can pass quietly through the later agents. Take our refund example. The order-lookup tool returns an old price. Every later agent does its job correctly with that price. The customer then gets a polite message with the wrong refund, and no error appears anywhere. Today, developers read the logs by hand. They usually blame the step where the error showed up, not where it started. They then trust a fix after one re-run, which a random model can pass by luck.

What we are building. Granite is a tool for investigating these failures. Its tagline is "find the failure, prove the cause, verify the fix." It runs a workflow and checks the result and each step against fixed rules and trusted data. If a finished run is wrong, it opens a finding. It then follows the data backwards to rank where the error began. To test the top suspect, it re-runs the workflow five times with that step fixed and twice without the fix. A fixed rule then gives the verdict: CONFIRMED, REFUTED or INCONCLUSIVE. A person accepts or rejects the finding. All evidence is stored safely and can be downloaded as a report. Granite runs for free on local or recorded models, so private data stays with the team.

What changed after the proposal. The panel asked for a better interface, clearer features and inputs, a clear way to verify results, and a redefined workflow. We saw that all five comments had one cause. We had designed mechanisms, not a workflow for the user. So we rebuilt the product around one workflow and one result, the "verified finding." We replaced the read-only demo console with a working application. We also found close prior work, such as DoVer and some commercial tools, and adjusted our claims. Our contribution is now three things: detection through automatic checks, tracing the cause through the data flow, and verification by re-runs with controls. All of it runs on our own machines.

Current state. Iteration 1 is built and running online at https://granite-demo.onrender.com. Eight features work end to end. All 381 automated tests pass, and tests run automatically on every change. All 21 design diagrams are generated from the code. We also ran a ten-task test with targets set in advance. It met all six targets. It even caught a real model error that

nobody planted. It also showed one design limit, two faults in a row, which we have recorded.

Risks and gaps. The code was built quickly with AI help and pushed from one account. Each member now has to show they own their module and work through reviewed pull requests. Personal data is not yet hidden before it is stored. Our test covers only one fault type and a small sample. We have not yet tested the interface with outside users.

Remaining work. Iteration 2, up to FYP-1 Final, adds trace import, a second workflow and three fault types. It also adds a Python SDK with a LangGraph adapter, fix suggestions, data hiding, PDF reports and a first comparison against other methods. FYP-2 adds a benchmark of at least 300 runs, a CrewAI adapter and full user testing.

Contents

List of Figures

List of Tables

List of Abbreviations

AI	**Artificial Intelligence**
API	**Application Programming Interface**
AST	**Abstract Syntax Tree**
CCP	**Complex Computing Problem**
CD	**Continuous Delivery / Deployment**
CI	**Continuous Integration**
DEI	**Diversity, Equity and Inclusion**
DFD	**Data Flow Diagram**
DSL	**Domain-Specific Language (Granite's check language)**
ERD	**Entity Relationship Diagram**
FR	**Functional Requirement**
FYP	**Final Year Project**
JSON	**JavaScript Object Notation**
LLM	**Large Language Model**
MAS	**Multi-Agent System**
NFR	**Non-Functional Requirement**
PII	**Personally Identifiable Information**
PoC	**Proof of Concept**
PR	**Pull Request**
SDK	**Software Development Kit**
SRS	**Software Requirements Specification**
SSD	**System Sequence Diagram**
SUS	**System Usability Scale**
UAT	**User Acceptance Testing**
UI / UX	**User Interface / User Experience**
VR-1	**Verdict Rule, version 1**
WCAG	**Web Content Accessibility Guidelines**

Chapter 1

Introduction and Project Vision

This chapter introduces Granite. It explains the problem of silent failures in multi-agent LLM workflows and why it matters. It then sets out our goals, scope, modules and users, how we measure success, and the limits and ethical points we must respect.

1.1 Project Overview

Granite is a self-hosted tool for investigating failures in multi-agent workflows built on large language models. Its tagline sums up its purpose: find the failure, prove the cause, verify the fix.

Granite runs a workflow in which several AI agents work together. It checks whether the result is really correct and finds the step where the error first entered. It proves that this step caused the failure by re-running the workflow under control. Finally, it records a person's decision along with all the evidence.

The main output is a verified finding. This is a recorded failure with its symptom, the suspected cause, evidence that can be reproduced, a verdict from the re-runs, and the reviewer's decision.

Our first test system is the Refund Desk, a five-agent customer-support workflow that uses public retail data. The project is research-based (Stream A). It starts from the ErrorProbe work on finding the cause of failures, and the gap its authors point out. We turn that into a working product and measure how well it performs.

1.2 Problem Statement

Multi-agent LLM workflows can fail without any error. An early wrong value is passed on by later agents that work correctly. The run ends with a confident but wrong answer.

We call this a silent failure: the run finishes normally, shows no error, but gives a wrong answer.

Teams usually notice it only when a customer complains. They read logs by hand and often blame the wrong step. They also trust a fix after one re-run, which is weak proof for a random model.

Current methods find the faulty step with about 60% accuracy at best. They also expect failures to leave a visible sign. No self-hosted tool finds silent failures, locates their start and proves the cause.

Terms used in this report.

- **Failure: a run with a wrong final answer.**
- **Fault: the defect behind it, such as an old price from a tool.**
- **Error: the wrong value the fault creates.**
- **Origin: the step where the error first appears.**
- **Symptom: the step where the failure becomes visible.**
- **Explicit failure: one the system reports, such as a crash or timeout. A silent failure is not reported.**

1.3 Problem Context and Motivation

Consider a customer who asks a retail support system to return two delivered items. Five agents work together:

- **Intake reads the request.**
- **Order Lookup fetches the order and its prices.**
- **Policy decides if the refund is allowed.**
- **Refund Calculator adds up the refund.**

- **Writer replies to the customer.**

Suppose the lookup tool returns a price that is $0.25 out of date. Every later agent still does its job correctly. The customer gets a polite message, but the refund is $0.25 short. The run shows "completed" and no error appears anywhere. The problem shows up at the last step, but its cause is three steps earlier.

Workflows like this are now used in customer support, coding assistants and research tools. There, a wrong answer can move money or mislead people.

Commercial monitoring tools record runs well, and some now suggest causes using AI. But these suggestions are just one model's opinion. The tools are paid cloud services, so some teams cannot send client data to them. They also do not publish how accurate they are.

Our aim is to give developers and reviewers a result they can trust and defend. It should rest on automatic checks and controlled re-runs, not one model's opinion. It should also be free and run on the team's own machines.

Figure 1.1 shows the workflow and the data each step reads; the stale-price fault enters at Order Lookup and becomes visible only in the Writer's message.

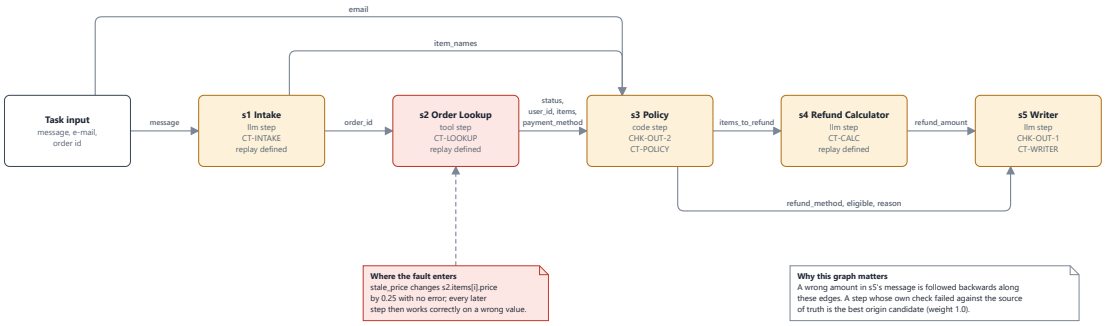


Figure 1.1: The Refund Desk workflow: five agents, the data each step reads, and the seven checks

1.4 Project Vision

When a multi-agent AI workflow gives a wrong result, even without an error, Granite tells the team where it went wrong. It proves the cause and shows whether a fix really works. All of this runs on the team's own machines.

Granite is built for developers and QA reviewers of multi-agent LLM workflows. They need to know why a run went wrong and whether a fix is real. Granite turns each suspected failure into a verified finding.

Other debuggers mostly rely on a language model's opinion of the logs. Granite instead uses automatic checks and controlled re-runs. It is also free, running on local or free models.

Five principles guide the design:

- **Evidence comes before opinion.**
- **Every finding is recorded.**
- **Say "inconclusive" when a result is unclear.**
- **A person makes the final decision.**
- **Every core feature works offline.**

1.5 Project Objectives

1. **Detect silent failures with automatic checks against trusted data, without false alarms on clean runs.**
2. **Find the step where a failure began by following the data flow, and rank that step first when it leaves clear evidence.**
3. **Prove the suspected cause by re-running the workflow several times with the step fixed and a few times without it, and decide with a fixed rule. Report "inconclusive" when the result is unclear.**
4. **Store every finding as evidence that cannot be secretly changed, with a human review and a downloadable report.**
5. **Deliver it as a working product that is online, easy to use, automatically tested and free to run.**
6. **Test the system on a labelled set of runs, including clean ones, and compare it with other methods (from FYP-1 Final onward).**

1.6 Project Scope

In scope:

- **Running multi-agent workflows, with an option to inject faults.**
- **Importing runs from other tools (Iteration 2).**
- **Automatic checks that open findings for silent failures.**
- **Tracing a failure back to where it began.**
- **Proving the cause with controlled re-runs.**
- **Human review, evidence, reports and history.**
- **A Fault Lab and an evaluation module.**
- **A Python SDK with LangGraph and CrewAI adapters.**
- **Running on the team's own machines with local or recorded models.**

Out of scope for the whole FYP:

- **A paid, multi-customer online service.**
- **Kubernetes or message queues.**
- **Applying fixes automatically to a live system.**
- **Training our own models.**
- **Testing on seven frameworks.**
- **A classifier that predicts failures in advance.**
- **Real customer data on the public demo.**

Scope table

Table 1.1: Project Scope

Module	In / out at Mid-1	Proposal origin	Mid-1 status	Final-1 target	Success metric
M1 Run	In	Fault-injection harness	Done (Refund Desk)	+ second workflow, 3 fault types	Runs recorded completely; reproducible in recorded mode
M2 Import	Out	SDK and adapters	Not started	Trace import, SDK + LangGraph	Imported runs validated with error paths
M3 Check	In	Semantic Error Diagnostic	Done	+ check editor	Detection $\geq 90\%$, false alarms 0 on clean runs
M4 Trace back	In	Culprit Agent, Error Step Localisation	Done	+ cycle bound	Origin ranked first $\geq 80\%$ ($\geq 95\%$ over 20 runs as acceptance)
M5 Verify	In	Verification Agent, Verified Automatic Repair	Done	+ resume	Verdict precision $\geq 85\%$ (evaluation)
M6 Review & Resolve	In (review)	Repair Agent, Recovery Controller, Repair Memory	Review done	Fix suggestion, recovered run	Every decision recorded with reviewer and time
M7 Report	In (basic)	Dashboard	MD/JSON done	PDF, regression export	Every finding carries ≥ 1 reproducible evidence item
M8 Fault Lab & Evaluation	Out	Cross-architecture benchmark	Feasibility run only	Benchmark v0 ≥ 120 runs, baselines	Metrics with confidence intervals

1.7 Project Modules

The ten mechanisms approved at the proposal were mapped onto eight modules that users can understand. This keeps the approved idea while making it clearer.

- **Run runs a workflow on the user's input, optionally with an injected fault, and records every step.**
- **Import brings in runs from other tools through file upload, an SDK and framework adapters.**

- **Check runs automatic checks on every run and turns silent failures into findings.**
- **Trace back ranks the steps where the failure may have started, by following the data. An optional AI second opinion is shown, marked as unverified.**
- **Verify tests a suspected cause, or a proposed fix, with controlled re-runs.**
- **Review & Resolve records the human decision. Later it will add fix suggestions, recovered runs and a library of known fixes.**
- **Report exports findings, regression tests and history.**
- **Fault Lab & Evaluation creates labelled runs, including clean ones, and compares the system with other methods.**

1.8 Stakeholders and User Classes

Table 1.2: Stakeholders and User Classes

User class	Context	Goal with Granite	Key need
Agent developer (primary)	**Builds agent workflows at a software house, often for clients whose data cannot go to external services; small budget**	**Find why a run went wrong and prove a fix before shipping**	**Fast localisation, trustworthy verification, self-hosting**
QA / reliability reviewer (primary)	**Reviews incidents and signs off fixes**	**Decide accept or reject on clear evidence**	**Plain-language evidence, verdicts, an audit trail**
Student / researcher	**Builds multi-agent projects without paid tools**	**Learn why agents fail; test robustness**	**Free, local, explains failure modes**
Auditor / client representative (secondary)	**Receives incident reports**	**Proof of what failed and what was done**	**Exportable, tamper-evident reports**

Granite has other stakeholders as well:

- **End customers of the agent system, who need correct outcomes.**

- **Privacy and compliance roles, who care about redaction, data retention and self-hosting.**
- **The supervisor and panel, who set the academic standard and evaluate the work.**
- **The team, who each need clear individual ownership they can defend.**
- **External parties such as model providers and framework maintainers, because of licences and rate limits.**

We plan four to six interviews with practitioners from Week 9 to check these needs.

1.9 Success Criteria

Table 1.3: Success Criteria

Area	Criterion	Mid-1 result
Detection	**Injected faults detected in $\geq$ 90% of faulty runs; 0 false alarms on clean runs**	**10/10 detected; 0 false alarms**
Attribution	**True origin ranked first in $\geq$ 80% of runs**	**10/10**
Verification	**True origin CONFIRMED in $\geq$ 80%; innocent step REFUTED in $\geq$ 80%**	**9/10 and 10/10**
Reliability	**$\geq$ 95% of runs complete**	**20/20**
Engineering	**All automated tests pass; static analysis clean; CI green**	**381/381; clean; green**
Product	**Core journey works live with failure cases; deployed URL**	**Done; live on Render**
Usability (Final-1)	**Task success $\geq$ 80% unaided; SUS $\geq$ 68**	**Protocol ready; sessions pending**
Evaluation (Final-1/Mid-2)	**Measured against last-agent and LLM-judge baselines with confidence intervals**	**Planned**

1.10 Constraints and Assumptions

Constraints:

- **No recurring cost. We use only local models, recorded answers and free hosting tiers.**

- **Small local models (llama3.2:3b on a CPU). They are slow and make mistakes.**
- **Free hosting sleeps when idle and resets its disk.**
- **The team has three people, all taking other courses.**
- **The academic calendar is fixed.**

Assumptions:

- **A workflow can declare which earlier outputs each step reads. This lets us record the data flow instead of guessing it.**
- **There is a source of truth for the values we check. Here, that is the order and catalogue data.**
- **Replays can be made repeatable by fixing seeds and recording model answers.**
- **Checks written by the operator capture what "correct" means.**

Granite can only detect failures that some check covers. This is a stated limit. It is not part of what we define as a failure.

1.11 Ethical, Legal and DEI Considerations

Ethical. Granite only gives advice. A human reviews every finding, and the system never applies a fix to a live system by itself. We inject faults only into our own workflows, and later into those of test participants who have agreed to it. Trace data from a real system would be redacted, kept for a limited time and self-hosted. Any real user's data would need informed consent first. The public demo uses only synthetic data.

Legal. We record the licence of every dataset and component:

- **tau-bench retail data and Who&When are under the MIT licence.**
- **Llama 3.2 is used under Meta's community licence. We distribute only its recorded answers.**
- **Every software dependency has a permissive licence.**

We have not yet chosen a licence for Granite itself.

DEI. Most automated failure diagnosis today is sold as a paid cloud service. Granite runs on local models at no cost. This gives students, small teams and researchers

with fewer resources the same tool as well-funded teams. The interface never shows status by colour alone, and automated accessibility checks cover every screen. One limit: the workflows and data are in English only, so our results do not carry over to traces in other languages. The project relates to SDG 9.5 (strengthening technological capability). We give this as context, not as a claimed contribution.

Chapter 2

Research and Existing Solutions

This chapter reviews the research and products that diagnose failures in multi-agent systems. It compares them on what matters for silent failures and identifies the gap Granite fills. It then shows how what we found shaped the design. It also explains why this is a complex computing problem.

2.1 Research Method / Approach

1. **Papers. We read seven papers on multi-agent failure attribution in full, not just the abstracts. These were Who&When (ICML 2025), AgenTracer (ICLR 2026), Scope Delineation Before Localization (SDBL, AAAI 2026), TraceElephant (ACL 2026), ErrorProbe (ACL 2026 Findings), AgentAsk (ACL 2026) and PROTEA (ACL 2026 Demonstrations). After the proposal we added an eighth, DoVer (arXiv preprint, 2026). For each paper we noted the dataset, the ground truth, the reported accuracy and the limits the authors themselves state.**

2. **Products. We compared commercial and open-source tools using their public documentation.**

3. **Past FYPs. We checked the department's full FYP archive for overlap. It has 660 titles, from Spring 2019 to Fall 2025.**

4. **Spikes. We ran eight proof-of-concept spikes to test our riskiest assumptions before fixing the design.**

2.2 Existing Solutions / Products

Research systems.

- **Who&When (1) defined automated failure attribution and released a benchmark labelled by humans. Its best method found the right step only 14.2% of the time.**
- **AgenTracer (2) created training data by injecting faults and replaying runs, then trained an 8B model. It found the right step about 21% of the time on hand-crafted logs and 37–43% on automated ones.**
- **SDBL (3) first marks out the scope of a failure, then finds it. It reached 26–33% step accuracy.**
- **TraceElephant (4) showed that much of the difficulty comes from incomplete traces. With full traces, step accuracy rose by up to 76%.**
- **ErrorProbe (5) uses a failure taxonomy, traces backwards from the symptom, and checks its answer with a team of LLM agents and a verified memory. Its best step accuracy was 59.5%, and its average was about 43%.**
- **AgentAsk (6) steps in at hand-offs between agents to prevent errors. It does not diagnose them.**
- **PROTEA (7) is an interactive tool. It suggests prompt fixes and re-runs the workflow once a developer accepts one.**
- **DoVer (8) is the closest work to ours. An LLM proposes where a failure started. DoVer then edits that step and re-runs the workflow three times. The hypothesis counts as validated if at least two of the three runs succeed. This turned 18–28% of failed runs into successes on GAIA and AssistantBench, and 49% on GSMPlus.**

Products.

- **LangSmith Engine (9) groups production failures, finds root causes in the code and drafts fixes. It only works inside LangSmith.**
- **Patronus Percival (10) uses LLM judges to find over twenty failure types in traces and suggests prompt fixes.**
- **Galileo's Insights Engine (11) finds failure types and links them to spans.**

- **Raindrop (12) detects silent failures in live agents from conversation signals. It offers experiments to test fixes.**
- **Invariant Labs' analyzer (13) runs rule-based checks on agent traces, with a focus on security.**
- **Langfuse, Arize Phoenix, Braintrust, Weights & Biases Weave and Opik capture traces and run evaluations. Diagnosis is left to people.**

Prior FYPs. None of the 660 archived titles tackles this problem. Multi-agent systems appear only as the product being built, such as an agent team that does a task. None treats them as something to diagnose. The nearest project, from Spring 2024, detected hallucinations in a single model's text. It had no multi-agent attribution and no verification.

2.3 Comparison of Existing Solutions

Table 2.1: Comparison of Existing Solutions

Solution	Type	Detects silent failures?	Finds origin step?	Verifies cause by re-running?	Handles model randomness?	Self-hosted, $0?	Accuracy published?
Who&When	**Research**	**No**	**Yes (LLM)**	**No**	**No**	**Research code**	**Yes (14.2% step)**
AgenTracer	**Research**	**No**	**Yes (trained model)**	**Only to label data**	**No**	**Partly released**	**Yes (~21–43% step)**
ErrorProbe	**Research**	**No (stated limit)**	**Yes (LLM team)**	**Tool-based checks**	**No**	**Research code**	**Yes (~43% avg, 59.5% best)**
DoVer	**Research (preprint)**	**No**	**Yes (LLM)**	**Yes (intervention)**	**3 runs, 2-of-3 rule; no controls**	**Research code**	**Flip rates (18–28%; 49% on GSMPlus)**
PROTEA	**Research demo**	**No**	**Node level**	**Re-runs after a human fix**	**No**	**Closed**	**Case studies only**
LangSmith Engine	**Commercial**	**Partly**	**Yes (LLM)**	**Reopens on regression**	**Not documented**	**No (SaaS)**	**No**
Patronus Percival	**Commercial**	**Failure types via LLM judges**	**Yes (LLM)**	**Not documented**	**No**	**No (SaaS)**	**No**
Raindrop	**Commercial**	**Yes (conversation signals)**	**Not per step**	**Experiments**	**Not documented**	**No (SaaS)**	**No**
Invariant analyzer	**Open source**	**Rule-based checks**	**No**	**No**	**n/a**	**Yes**	**n/a**
Granite	**This project**	**Yes (executable checks + source of truth)**	**Yes (deterministic provenance)**	**Yes (N replays + M controls)**	**Yes (controls, verdict rule, INCONCLUSIVE)**	**Yes**	**Pre-registered; benchmark planned**

2.4 Limitations and Gaps

Three gaps come up again and again.

1. **Silent failures. Attribution methods assume a failure leaves a visible anomaly. ErrorProbe's own limitations section names silent failures as a case it cannot handle. These are outputs that look valid but are wrong in meaning. It suggests "test-time oracle feedback" as future work.**
2. **Verification without controls. Some tools re-test a fix only once (PROTEA). DoVer repeats each fix three times, which helps. But no tool also re-runs the workflow *without* the fix. Without these control runs, a success caused by luck looks the same as a real fix.**
3. **Trust and access. Commercial diagnoses are language-model opinions with no published accuracy. They run as cloud services, which some teams cannot use with client data.**

Benchmarks in this field also contain only failed runs, with no clean controls. So false-alarm rates are never measured.

2.5 Proposed Solution

Granite is a self-hostable product that targets these gaps.

1. **Detect. It finds silent failures with executable checks, written by the operator, against an independent source of truth. These are the test-time oracles that ErrorProbe lists as future work.**
2. **Trace back. It finds the origin by following how values flow through the run. This is deterministic, with no language model in the core path.**
3. **Verify. It tests each hypothesis with N replays that apply the fix and M control replays that do not. A fixed, versioned verdict rule decides the result. Every artefact is stored as hashed evidence for a human reviewer.**

We will evaluate Granite on a labelled fault-injection benchmark that includes clean runs. We will compare it with last-agent and LLM-judge baselines. We will measure origin accuracy, false-alarm rate, verdict precision and recovery rate.

We no longer claim to be "first", or that "nobody verifies fixes". DoVer and several products already re-run or experiment. Our contribution is the combination:

executable-check detection, deterministic attribution and verification with control runs. It is measured honestly and runs at zero cost.

2.6 Complex Computing Problem

Table 2.2: Complex Computing Problem

Seoul Accord characteristic (14)	How Granite meets it
1. Conflicting requirements	Confidence versus replay cost and time; privacy (local model) versus model quality; sensitivity versus false alarms
2. No obvious solution	The best published attribution finds the right step at most about 60% of the time, and about 43% on average; a re-run without controls can pass by luck when the model is random
3. Depth of knowledge	Graph and provenance algorithms, statistics (intervals, controls), LLM behaviour, sandboxed evaluation
4. Unfamiliar issues	Re-running something that is not deterministic; failures that show up several steps after their cause
5. Beyond standard practice	A tracing library or a CRUD app cannot solve it
6. Diverse stakeholders	Developers, reviewers, end customers, auditors, privacy roles (to be confirmed by interviews)
7. Significant consequences	Wrong refunds, wrong advice, money moved on wrong numbers
8. Interdependence	Failures spread through dependent agents and steps
9. Ill-defined cause	At diagnosis time we do not know if there is a failure, where it began, or what the right value is

2.7 Technical Challenges

- **Repeatable replays. The model is random, so a verdict means little unless replays can be repeated. We use recorded answers, fixed seeds and control replays.**

- **Safe checks. Users write check expressions, and these must not run arbitrary code. We wrote our own evaluator that only accepts a whitelist of syntax.**

- **No leaked answers. The analysis must not see the injected fault's ground truth. A test enforces this.**
- **Partial re-runs. Earlier outputs must stay fixed while only the later steps run again.**
- **Small local models. They are slow (5–35 seconds per call on a CPU) and make arithmetic mistakes.**
- **Docs drifting from code. The code changes quickly. We generate diagrams from the code and check them in CI.**

2.8 Research Findings and Design Decisions

Table 2.3: Research Findings and Design Decisions

Finding (source)	Design decision
LLM-based attribution finds the right step only 14–60% of the time (Who&When, AgenTracer, SDBL, ErrorProbe). Our own test with a 3B model got 0–25% over five runs (Spike 1)	**Core attribution uses deterministic provenance. An LLM opinion is optional and marked unverified (ADR-005)**
ErrorProbe lists silent failures and test-time oracles as open problems	**Detection uses executable checks against a source of truth, not an answer key (ADR-004)**
One re-run can pass by luck. DoVer repeats three times but has no control runs. Our replay spike worked, but on very few runs (Spike 5)	**Verification uses N = 5 fix replays and M = 2 controls under a fixed rule, VR-1 (ADR-006)**
Small models vary a lot from run to run (Spike 1, repeated five times)	**We report ranges and intervals, and set thresholds in advance**
TraceElephant: incomplete traces limit diagnosis	**Every step records its full inputs, outputs and declared reads (Granite Trace v1, ADR-009)**
Fault injection gives exact ground truth (AgenTracer; Spike 3b)	**Faults are rule-based. Their ground truth is stored but hidden from attribution**
A preventive classifier was close to random (Spike 7)	**Dropped from scope and reported as a negative result**
DoVer and commercial tools already re-run or diagnose	**Novelty restated as checks + provenance + controls + self-hosting**

Finding (source)	Design decision
Cost estimated from measured token counts	**Zero-cost design: local Ollama model and recorded answers (ADR-008)**

Chapter 3

Proposal Feedback and Iteration Plan

This chapter covers the feedback from the proposal defense and how we acted on each comment. It then sets out the iteration plan: what Iteration 1 delivered, what Iteration 2 will add and who does what. It ends with our progress against the plan, including delays and how we are recovering.

3.1 Proposal Feedback Summary

Summarize feedback received during the proposal stage.

The proposal was approved at the defense on 11 September 2026. The panel made five comments:

1. **“UI/UX needs improvement.”**
2. **“Features are not sufficiently clear.”**
3. **“It is unclear what inputs each feature will accept.”**
4. **“It is unclear how identified flaws/issues will be verified and how verification results will be captured.”**
5. **“The overall project workflow must be redefined.”**

We also noted a sixth, unspoken point: the proof of concept showed research results, not a product.

Looking at the comments together, we saw one root cause. We had designed mechanisms, not a workflow for a user. The proof-of-concept console only displayed spike

results recorded earlier. There was no screen where a user could give Granite something and get a verified result back. So we rebuilt the product around one workflow and one main output: the verified finding.

3.2 Proposal Action Register

Table 3.1: Proposal Action Register

No.	Feedback / Required change	Action taken	Status	Evidence
A1	**UI/UX needs improvement**	**Replaced the PoC console with five task-based screens (investigations, new run, run, finding, guide). Added a four-stage stepper, loading, empty and error states, light and dark themes, a phone layout and a keyboard-operable graph. Added automated accessibility checks and a before/after comparison**	**Partly done (usability test with outside users pending)**	**Screenshots and before/after record in `evidence/demos/`; accessibility audit**
A2	**Features not sufficiently clear**	**Turned ten mechanism names into eight plainly named features in one fixed order. The same list is a page in the product**	**Done**	**Feature catalogue; in-app "How it works" page**
A3	**Unclear what inputs each feature accepts**	**Every input has a type, a limit and a refusal message. The server enforces these and tests cover them. Trace schema and API docs published**	**Done**	**Data dictionary; validation tests; `/docs`**

No.	Feedback / Required change	Action taken	Status	Evidence
A4	Unclear how flaws are verified and results captured	Verification levels V0–V3. Replay with N fixes and M controls under rule VR-1, with a Wilson interval. Hashed, append-only evidence; review records; exportable report	Done	Verification sequence diagram; decision-table tests; feasibility results
A5	Overall workflow must be redefined	One workflow (run, check, detect, trace back, verify, review, resolve, record). A state machine enforces it and a stepper shows it	Done (change record C-01 awaits signature)	State machine; flow and activity diagrams
A6	PoC showed research, not a product	A working vertical slice with CI/CD, deployed publicly	Partly done (CD hook and tag pending)	Live URL; CI and CD runs; deployment record

Figure 3.1 compares the screens shown at the defense with the product at Mid-1.

3.3 Iteration Strategy

We work in iterations, and every iteration ends with a working product we can demonstrate, not just documents. Iteration 1 builds one complete vertical slice: a single workflow and a single fault, taken through every stage from run to report. This proves the whole investigation loop before we add breadth. Later iterations add:

- new inputs (trace import and the SDK);
- more workflows and faults;
- repair and recovery;
- the evaluation.

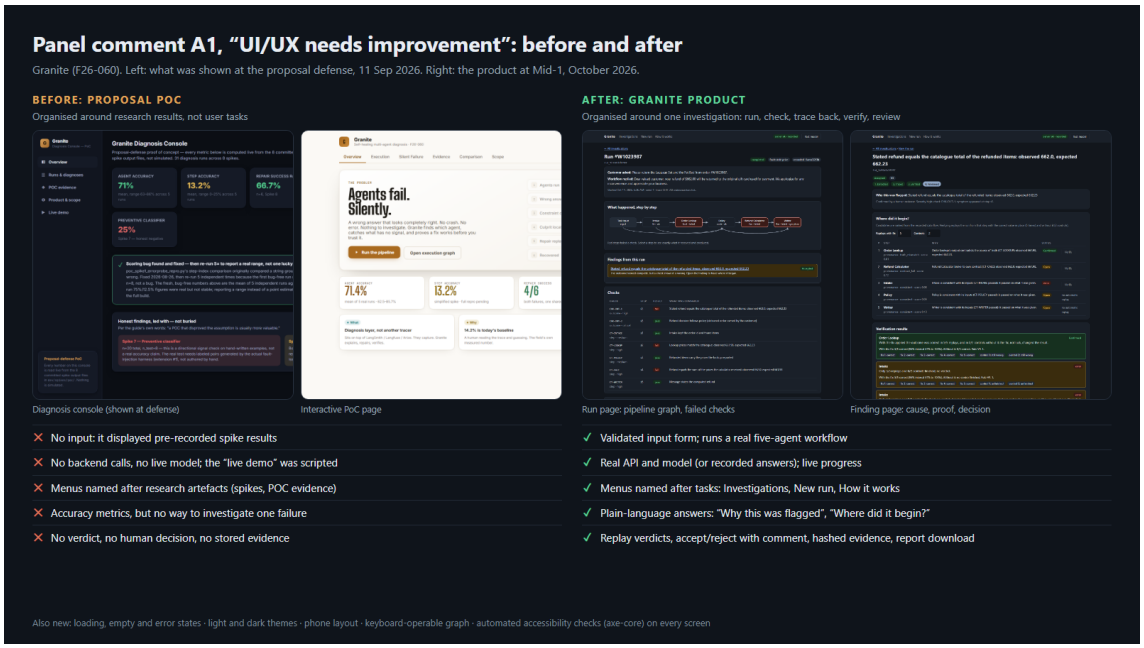


Figure 3.1: Panel comment A1: the proposal-defense PoC (left) compared with the Mid-1 product (right)

Each iteration gets a release tag. We set scope per milestone using MoSCoW. Any change to scope or design goes in a change log before results are known.

3.4 Iteration 1 Plan

Objective: a working core investigation slice for Mid-1 (release v0.1.0).

Use cases: UC-03 run a workflow, UC-05 inspect a run, UC-06 analyse a run, UC-07 investigate a finding, UC-08 verify a hypothesis, UC-09 review a finding and UC-12 browse history. UC-11 export report is a Should.

Deliverables:

- **the core library (trace, runner, check language, checks, attribution, verification, lifecycle, evidence);**
- **the Refund Desk workflow on real data with a real local model;**
- **the API, with a database and background jobs;**
- **the web interface and CI/CD;**
- **a pre-registered feasibility run;**

- **the design artefacts from handbook Table 1a, system tests and manuals.**

Evidence: a live demo, automated test results, CI runs, generated diagrams and the feasibility results.

3.5 Iteration 2 Plan

Objective: breadth and new inputs, reaching FYP-1 Final (release v0.2.0) with about half of the committed effort built.

Use cases: UC-01 configure a project, UC-02 define and test checks, UC-04 import a trace, UC-10 suggest and verify a fix, UC-11 export a report and regression test, UC-13 Fault Lab, UC-14 evaluation v0 and UC-15 SDK with LangGraph.

Deliverables:

- **trace import, a second workflow and three fault types;**
- **a check editor and PII redaction;**
- **fix suggestion and recovered runs;**
- **PDF reports, the SDK and a LangGraph adapter;**
- **a benchmark of at least 120 runs, with baselines;**
- **a usability pilot and an install by someone outside the module's owner.**

3.6 Work Breakdown and Responsibilities

Table 3.2: Work Breakdown and Responsibilities

Member	Area	Iteration 1 modules	Iteration 2 work
Shahoud Shahid	Workflows, Data & Experience	Runner, Refund Desk workflow, faults, LLM providers, web interface shell	Trace import, more fault types, redaction, SDK and LangGraph adapter, Fault Lab, help page

Member	Area	Iteration 1 modules	Iteration 2 work
Muhammad Noor Ul Haq	Investigation Engine	Check language and checks, attribution, feasibility evaluation	Check editor, attribution cycle bound, fix D-25, second workflow's checks, evaluation harness and baselines, evaluation screen
Saif Shahzad	Verification & Platform	Verification and verdict rule, lifecycle, API, database, jobs, CI/CD	Experiment resume, fix suggestion and recovered run, regression export, PDF reports, release pipeline, non-owner install

3.7 Progress Against Plan

Iteration 1 items 1 to 13 were finished between 6 and 11 October. They cover planning, the workspace, the core library, attribution, verification, the workflow, the API, the interface, the feasibility run, CI, the design artefacts, system tests and manuals, and the panel action register. The public deployment followed on 11 October. All 381 automated tests pass, and the feasibility run met every pre-registered threshold.

These items are behind schedule:

- **Fully automated deployment. The CD workflow runs, but its deploy step is waiting for a repository secret.**
- **Branch protection and the release tag.**
- **Pull requests from every member.**
- **The code walkthroughs.**
- **Redaction before storage.**
- **The usability check.**

3.8 Delays, Deviations and Recovery Plan

Delays. For three weeks after the defense there were no commits, while we redefined the workflow in response to the panel. We then built the product quickly, with AI assistance, between 6 and 11 October. Most of it was committed from one account.

So the work is not yet visible under its planned owners, and no member has yet walked through their code file by file.

Recovery.

- **Each owner does the code walkthrough for their module before the evaluation.**
- **From now on, each member contributes only through reviewed pull requests from their own account. The first pull requests were opened on 11 October.**
- **Before the meeting, a repository admin adds the deploy-hook secret, protects `main` and tags v0.1.0-mid1.**
- **The usability check runs in Week 9. Redaction is built in Week 10.**

Deviations from the plan, all recorded in the change log:

- **C-01: the workflow was redefined around the verified finding. It awaits the supervisor's signature.**
- **C-02: implementation decisions. These include a 120-second timeout for local models, one finding per failing outcome check, the policy step written as plain code, and redaction deferred.**
- **C-03: diagrams generated from code, simpler branching, an extra system-test level, a CD workflow, and enforced docstring and naming rules.**

Chapter 4

Software Requirements

Granite's requirements are grouped by product module. Each functional requirement has:

- **an ID;**
- **a priority (M = Must, S = Should);**
- **the milestone it is due at;**
- **an acceptance criterion that can be tested.**

Each Mid-1 requirement is traced to the code that implements it and the tests that check it. A script in CI fails if a traced file or test is missing.

4.1 User Interactions / Use Cases

The system has two main human actors and two external systems.

- **The Developer runs workflows and investigates them.**
- **The Reviewer decides whether to accept a finding. In a small team, one person may do both.**
- **The Administrator runs a self-hosted instance (FYP-2).**
- **The LLM Provider answers the workflow's language-model steps. It can be a local Ollama model, a hosted endpoint or recorded answers.**
- **An Instrumented Workflow sends its traces through the SDK (Iteration 2).**

Table 4.1: User Interactions / Use Cases

UC	Name	Actor	Milestone
UC-01	**Configure project (workflow, settings)**	**Developer**	**Mid-1 seeded; Final-1 editable**
UC-02	**Define, edit and dry-run checks**	**Developer**	**Final-1**
UC-03	**Run workflow with input (optional fault)**	**Developer**	**Mid-1**
UC-04	**Import trace file**	**Developer**	**Final-1**
UC-05	**Inspect run trace (graph, steps, checks)**	**Developer, Reviewer**	**Mid-1**
UC-06	**Analyse run (checks → findings → hypotheses)**	**System, included by UC-03/04**	**Mid-1**
UC-07	**Investigate finding**	**Developer, Reviewer**	**Mid-1**
UC-08	**Verify hypothesis by controlled replay**	**Developer, LLM Provider**	**Mid-1**
UC-09	**Review finding (accept / reject / request information)**	**Reviewer**	**Mid-1**
UC-10	**Suggest and verify a fix; produce a recovered run**	**Developer, LLM Provider**	**Final-1**
UC-11	**Export report / regression test**	**Developer, Reviewer**	**Mid-1 (report); Final-1 (regression)**
UC-12	**Browse and search history**	**Developer, Reviewer**	**Mid-1**
UC-13	**Generate labelled benchmark (Fault Lab)**	**Developer**	**Final-1**
UC-14	**Run evaluation against baselines**	**Developer (research)**	**Final-1 v0 / Mid-2**
UC-15	**Instrument own workflow via SDK**	**Developer, Instrumented Workflow**	**Final-1 (LangGraph), Mid-2 (CrewAI)**
UC-16	**Reuse a known verified fix**	**Developer**	**Mid-2**
UC-17	**Manage API keys, retention, delete data**	**Administrator**	**Mid-2**

Use cases in bold are built for Mid-1. The rest are planned.

4.2 Use Case Diagram

Figure 4.1 shows the Developer and Reviewer with the Mid-1 use cases.

- **Running a workflow (UC-03) always includes analysing the run (UC-06). This evaluates the checks, opens findings and ranks hypotheses.**
- **Investigating a finding (UC-07) extends to verification (UC-08) and review (UC-09). Verification involves the LLM Provider.**

Use cases planned for Iteration 2 and FYP-2 are drawn separately.



Figure 4.1: Use case diagram (implemented and planned use cases are drawn differently)

4.3 Detailed Use Cases

UC-03 Run workflow with input

- ***Actor:*** **Developer.**
- ***Precondition:*** **the Refund Desk workflow is registered and the server is reachable.**
- ***Main flow:***

1. **The developer opens New run. They load an example, or type a request, an e-mail and an order number.**
2. **They can choose the "stale price" fault and the model mode.**
3. **They press Run workflow.**
4. **The system checks the input, stores a queued run and returns a job.**
5. **The system runs the five steps. It records each step's inputs, outputs and timing, and applies the fault if one was chosen.**
6. **The system analyses the run (UC-06). It shows the run page with the pipeline graph, the check results and any findings.**

- ***Alternative flows:***
  - **4a. A field is invalid. The system refuses the run and shows one plain message per field.**
  - **5a. The model cannot be reached, or the task has no recorded answers. The run ends as "errored", with the reason and a "Run again" button.**
  - **5b. Too many requests in one minute. The system answers "too many requests" (429).**
- ***Postcondition:*** **a completed or errored run is stored with its trace and SHA-256 hash.**

UC-07 Investigate finding

- ***Actor:*** **Developer or Reviewer.**
- ***Precondition:*** **a finding exists.**
- ***Main flow:***
  1. **The user opens the finding from the run page or the investigations list.**
  2. **The system shows why it was flagged: the failed check, with its observed and expected values. It shows the finding's stage in the four-step stepper. Under "Where did it begin?" it lists the ranked hypotheses. Each has its step, evidence type, score and a plain reason.**
  3. **The user opens the evidence records to see the computation and whether its hash is valid.**
- ***Alternative flow:*** **2a. No step has direct evidence. Hypotheses are ranked by structure and marked as needing verification.**

UC-08 Verify hypothesis by controlled replay

- ***Actor:*** **Developer; LLM Provider.**
- ***Precondition:*** **a hypothesis exists for a step that has a repair rule.**
- ***Main flow:***
  1. **The user sets the number of replays with the fix (default 5) and the number of controls (default 2), then presses Verify.**
  2. **The system replaces the step's output with the corrected value. It keeps all earlier outputs fixed. It re-runs the later steps N times with the fix and M times without it.**
  3. **The system applies rule VR-1 to the pass counts. It shows the verdict (CONFIRMED, REFUTED or INCONCLUSIVE), the result of each replay and a 95% Wilson interval. The verdict is stored as evidence.**
  4. **The finding moves to the matching state.**
- ***Alternative flows:***
  - **2a. The step has no repair rule. The system refuses and says why.**
  - **2b. The replays need model answers that were not recorded. The experiment ends with no verdict and explains that a live model is needed.**
  - **3a. The verdict is REFUTED. The user verifies the next hypothesis.**

UC-09 Review finding

- ***Actor:*** **Reviewer.**
- ***Precondition:*** **the finding is CONFIRMED, INCONCLUSIVE or evidence-supported.**
- ***Main flow:***
  1. **The reviewer reads the evidence and the verdict.**
  2. **They choose Accept, Reject or Ask for information, and give a name and a comment.**
  3. **The system checks the comment rule and moves the finding to its new state. It records the review with the reviewer and time, adds the comment as evidence and writes an audit entry.**
  4. **After accepting, the reviewer can mark the finding resolved, or close it without a fix.**

- *Alternative flows:*
  - **2a. Rejecting, closing, or accepting an inconclusive finding needs a comment of at least five characters. Without one, the action is refused (422).**
  - **2b. The action is not allowed in the current state, for example accepting before verification. It is refused (409), and the allowed actions are listed.**

Figures 4.2 and 4.3 show the system sequences for running a workflow and verifying a hypothesis.

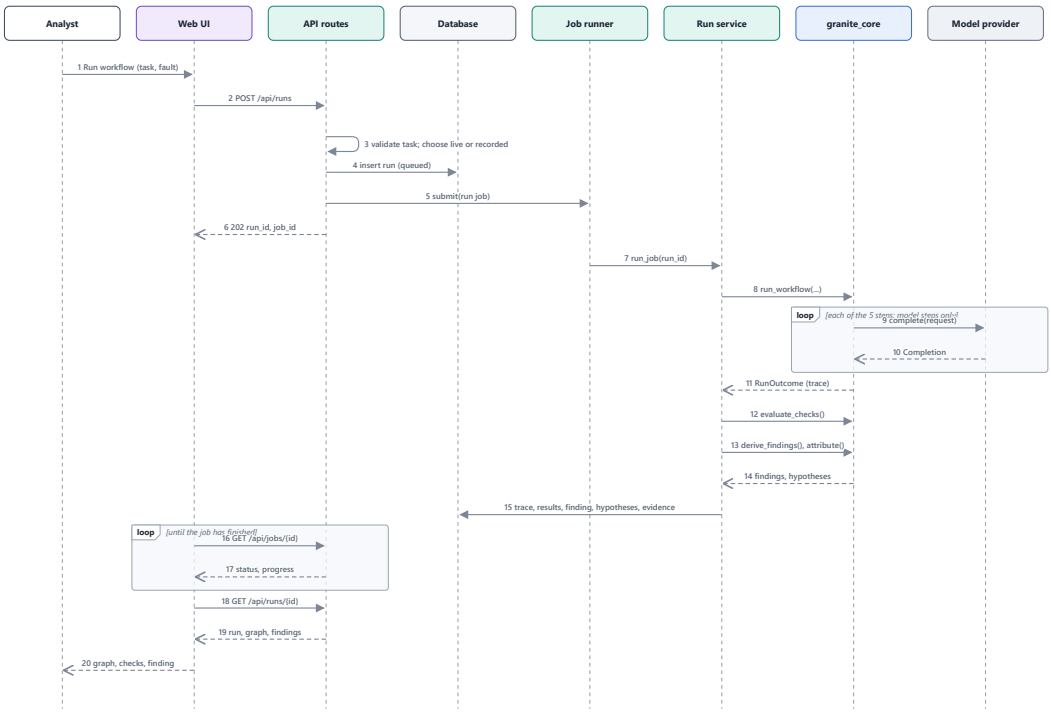


Figure 4.2: System sequence for UC-03, running a workflow

4.4 Functional Requirements

The functional requirements are grouped by product module. Each has an ID, a priority (M = Must, S = Should), a testable acceptance criterion and its status at Mid-1. Requirements for later milestones are in Appendix B.

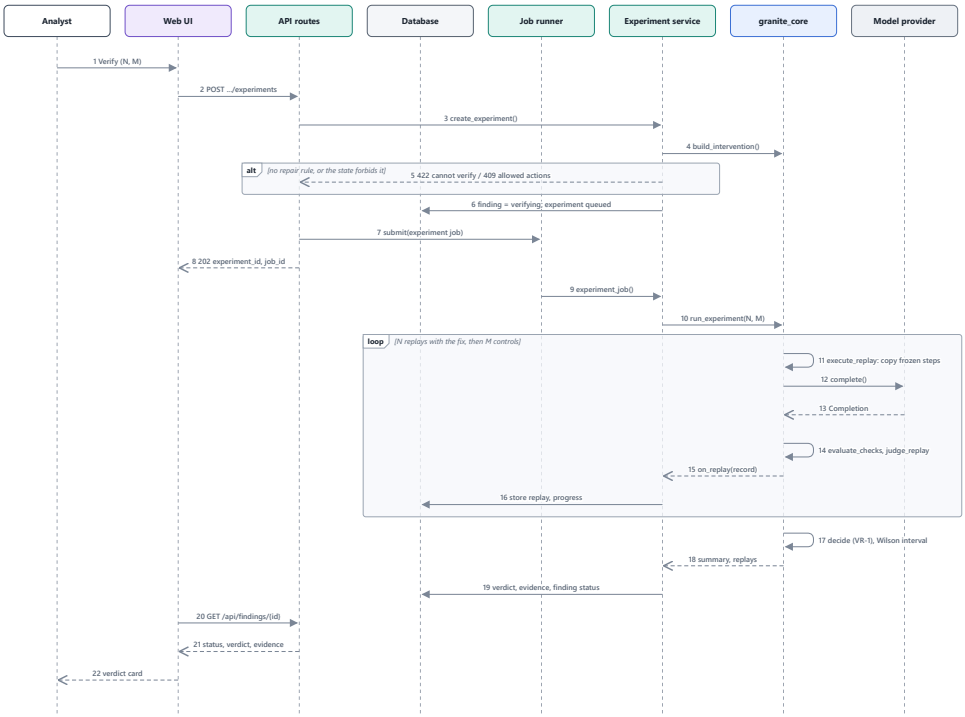


Figure 4.3: System sequence for UC-08, verifying a hypothesis by controlled replay

4.4.1 Module 1: Run

Table 4.2: Module 1: Run

ID	Requirement	Pri.	Acceptance criterion	Status
FR-RUN-01	**Run the Refund Desk workflow from a validated form. Record each step's inputs, outputs, model, parameters and timings**	**M**	**A valid run gives 5 steps with inputs, outputs and timestamps**	**Done**
FR-RUN-02	**Let the user turn on a named fault. Record its ground truth, but keep it hidden from attribution**	**M**	**The fault record exists; attribution cannot read it (test)**	**Done**
FR-RUN-03	**Support live and recorded model modes. Recorded mode replays saved answers**	**M**	**The same input in recorded mode gives byte-identical outputs over 3 runs**	**Done**

ID	Requirement	Pri.	Acceptance criterion	Status
FR-RUN-04	**Mark a run errored, with a useful message, when the model is unreachable or times out**	**M**	**With the port closed, the run ends errored within 35 s; the UI shows a message and a retry**	**Done (timeout 120 s for local models, deviation C-02g)**

4.4.2 Module 2: Check

Table 4.3: Module 2: Check

ID	Requirement	Pri.	Acceptance criterion	Status
FR-CHK-01	**Evaluate every check on every run. Store pass, fail, abstain or error, with observed and expected values**	**M**	**A fault run fails the outcome check and shows both values**	**Done**
FR-CHK-02	**Evaluate check expressions with a whitelisted syntax-tree evaluator. Reject anything not allowed**	**M**	**The malicious-expression set is rejected (25 tested; 15 required)**	**Done**
FR-CHK-03	**A missing field gives ABSTAIN, never PASS**	**M**	**A test with a missing field abstains**	**Done**
FR-CHK-04	**A completed run with a failing outcome check opens a silent-failure finding**	**M**	**Exactly one finding on the stale-price run**	**Partly (one finding per failing check, C-02h)**
FR-CHK-06	**Truth checks query source-of-truth functions on a fixed data snapshot. The query and answer are stored as evidence**	**M**	**Evidence holds the query, the response and the snapshot version**	**Done**

4.4.3 Module 3: Trace back

Table 4.4: Module 3: Trace back

ID	Requirement	Pri.	Acceptance criterion	Status
FR-ATT-01	**Give 1–5 ranked origin hypotheses per finding, each with a score, a reason and evidence**	**M**	**Order Lookup ranked first in $\geq$ 95% of 20 runs**	**Partly (10/10 measured; 20-run test not yet done)**
FR-ATT-02	**Compute scores from evidence weights and graph position, never from an LLM's own claim**	**M**	**The same input gives byte-identical output**	**Done**

4.4.4 Module 4: Verify

Table 4.5: Module 4: Verify

ID	Requirement	Pri.	Acceptance criterion	Status
FR-VER-01	**Run a verification under VR-1: keep earlier steps fixed, apply one change, re-run later steps N times plus M controls**	**M**	**Fixing the faulty step gives CONFIRMED, and the controls stay broken**	**Done**
FR-VER-02	**Decide the verdict with VR-1 and a Wilson 95% interval. Store the rule version**	**M**	**A table test over 20 (k, N, c, M) combinations**	**Done**
FR-VER-03	**A replay that causes a new failing check counts as a fail (regression guard)**	**M**	**A test with a new failure counts it as a fail**	**Done**

4.4.5 Module 5: Review, Report and History

Table 4.6: Module 5: Review, Report and History

ID	Requirement	Pri.	Acceptance criterion	Status
FR-REV-01	Accept, reject or ask for information, with comments. A comment is required to reject, or to accept an inconclusive finding. Store the reviewer and time	M	Reject without a comment is refused; the decision is stored	Done
FR-REV-02	Finding changes follow the state machine. Illegal changes are refused. Every change is audited	M	Accept before verification is refused (409)	Done
FR-REP-01	Export a finding report in Markdown and JSON with evidence hashes (PDF at Final-1)	S	The report downloads with hashes	Partly (PDF at Final-1)
FR-HIS-01	List runs and findings with filters and search, 25 per page	M	Filters by status, severity, level, date and text	Partly (no project filter; one project)
FR-SEC-01	Remove personal data before storing step inputs and outputs	S	No seeded personal data in the database after a run	Not met (D-07)

Appendix B lists the requirements for Iteration 2 and FYP-2. These cover import, the SDK, the check editor, fix suggestion, recovered runs, regression export, Fault Lab, evaluation, authentication and retention.

4.5 Non-Functional Requirements

These requirements set measurable qualities for the system. Each has a target and a way to measure it. Where we have checked it, its Mid-1 status is given.

4.5.1 Reliability

NFR-04. No unhandled error reaches the user. Each named failure ends in a documented state: model down, model timeout, bad input, check error, database unavailable and partial replay. At Mid-1, system tests SYS-ERR-01 to 07 and SYS-REL-01 to 03 cover model down, unrecorded tasks, illegal state changes, limits and recovery after a restart.

NFR-05. The deterministic parts (checks, attribution, verdict) give byte-identical output when run again. Golden-file tests check this.

4.5.2 Usability

NFR-09. A SUS score of at least 68, and at least 80% of core tasks completed without help. This is a pilot with 3 participants at Final-1; the protocol is ready.

NFR-10. WCAG 2.2 AA. Contrast is at least 4.5:1, the whole core journey works by keyboard, and status is never shown by colour alone. Automated axe-core checks pass on every screen, in light and dark themes.

4.5.3 Performance

- **NFR-01. Checks and attribution for a run of up to 50 steps finish within 2 seconds at the 95th percentile.**
- **NFR-02. A VR-1 verification with N = 5 and M = 2 finishes within 3 minutes with a hosted model. For a local 3B model on a CPU, we report the time but set no target.**
- **NFR-03. The first content appears within 3 seconds once the server is awake. Interactions respond within 200 ms.**

4.5.4 Security

- **NFR-07. Met:**
  - **bandit, pip-audit, npm audit and gitleaks report no high or critical issues on `main`;**
  - **the check language rejects every expression in the malicious set;**
  - **input and upload limits are enforced.**

- **NFR-08. No unredacted personal data in the database after a run. Not yet met. The public demo uses only synthetic data.**

4.6 Acceptance Criteria

Table 4.7: Acceptance Criteria

Feature	Acceptance check	Verified by	Result
Run	**A valid example run finishes with 5 recorded steps and a trace hash**	**SYS-CORE-01, test_runner**	**Pass**
Detect	**The stale-price run (one failing outcome check) opens one silent-failure finding; a clean run opens none**	**SYS-CORE-01, SYS-CORE-02**	**Pass**
Trace back	**Order Lookup is ranked first, with a truth-mismatch reason**	**SYS-CORE-01, feasibility M3 (10/10)**	**Pass**
Verify	**Verifying Order Lookup gives CONFIRMED (5/5 fixed, 0/2 controls); verifying the calculator gives REFUTED**	**SYS-CORE-03, feasibility M4/M5**	**Pass**
Review	**Accept before verification is refused; reject needs a comment; decisions are recorded**	**SYS-ERR-03**	**Pass**
Evidence	**Evidence is append-only and hashes match; ground truth never appears in evidence**	**SYS-SEC-02, SYS-SEC-03**	**Pass**
Errors	**Bad input is refused per field; model down ends errored with a reason**	**SYS-ERR-01, SYS-ERR-07**	**Pass**
History	**Runs and findings can be found again with filters**	**SYS-CORE-05**	**Pass**

4.7 Requirements Traceability

Table 4.8: Requirements Traceability

Requirement	Design element	Implementation	Test
FR-RUN-01	**Runner, Trace v1**	`runner.py::run_workflow`, `refund_desk/definition.py`	`test_runner.py`, `test_journeys.py`
FR-RUN-02	**Fault record hidden from analysis**	`faults.py::StalePriceFault`	`test_attribution_never_sees_fault_ground_truth`
FR-RUN-03	**Provider layer, cassette**	`llm.py::RecordedProvider`	`test_fr_run_03_recorded_mode_gives_byte_identical...`
FR-CHK-02	**Check language (ADR-004)**	`dsl.py::compile_expression`	`test_dsl.py::test_malicious_expressions_are_rejected`
FR-CHK-03	**Check results**	`checks.py::evaluate_check`	`test_missing_field_abstains_never_passes`
FR-ATT-01/02	**Provenance attribution (ADR-005)**	`attribution.py::attribute`	`test_attribution.py`, `test_attribution_ranks_the_injected_step_first...`
FR-VER-01/02	**VR-1 (ADR-006), replay plan**	`verification.py::decide`, `experiment.py::run_experiment`	`test_verification.py`, `test_experiment.py`
FR-REV-01/02	**Finding state machine**	`lifecycle.py::transition`, `services/reviews.py`	`test_lifecycle_...`, **SYS-ERR-03**
FR-REP-01	**Report service**	`services/report.py`	`test_journeys.py`

The full matrix for all 19 Mid-1 requirements is generated from the code by `scripts/check_traceability.py` (`docs/plan/requirements-trace.md`). See Appendix B.

Chapter 5

System Design

This chapter describes Granite's design: the layered architecture and its components, the interfaces between them, and the main architecture decisions with their reasons. It then covers how the design changed since the proposal, and the domain, data and behaviour models. All diagrams are generated from the code, so they match what is built.

5.1 System Architecture

Granite is a modular monolith: one repository, four packages, and dependencies that flow in one direction only.

- **The web interface (React) talks to the API over HTTP.**
- **The API (FastAPI) stores data, runs background jobs, and calls the workflow and the core library.**
- **The Refund Desk workflow depends only on the core.**
- **The core library (`granite_core`) holds the trace model, runner, check language, checks, attribution, verification, lifecycle and evidence logic. It has no web, database or network code.**

The core imports only the Python standard library and Pydantic. So its algorithms can be tested on their own in seconds. A script in CI fails the build if a forbidden import appears. External services, the language-model provider and the database, are reached only through the API layer.

Table 5.1: System Architecture

Layer	Package	May import	Must not import
Presentation	`apps/web`	**the API over HTTP**	**anything else**
Application	`apps/api` (`granite_api`)	**core, workflow, FastAPI, SQLAlchemy**	**UI code**
Workflow under test	`workflows/refund_desk`	**core**	**API, web or database libraries**
Domain core	`packages/granite_core`	**standard library, Pydantic**	**FastAPI, SQLAlchemy, HTTP clients, any workflow**

Figure 5.1 shows the layers and the direction of dependency.

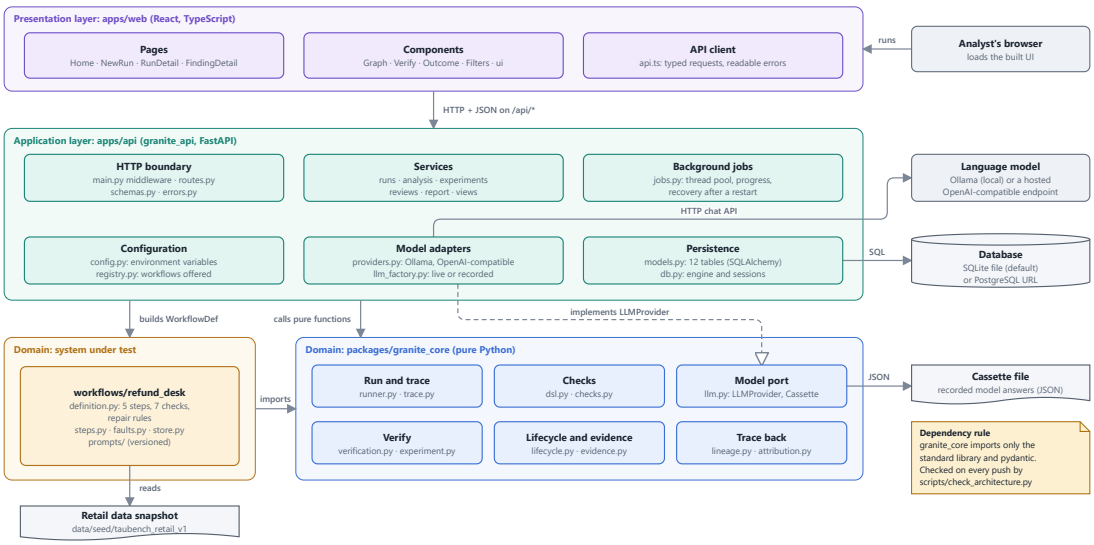


Figure 5.1: Layer diagram: one dependency direction, checked in CI

5.2 Architectural Components

- **Web interface (`apps/web`): five screens (Investigations, New run, Run, Finding, How it works), a keyboard-operable pipeline graph, the verification table and the review panel.**
- **API (`granite_api`): sixteen HTTP endpoints, request validation, RFC 7807**

error responses, security headers, a body-size limit, rate limiting and an optional access token.

- **Services: one per use case (runs, analysis, experiments, reviews, report).**
- **Job runner: a thread pool with a job table, so long runs and replays do not block requests. Interrupted jobs are marked failed on restart.**
- **Persistence: SQLAlchemy over SQLite. PostgreSQL is also supported.**
- **Provider layer: live (Ollama or an OpenAI-compatible endpoint), recorded (a cassette of real answers) or automatic.**
- **Core library: trace, runner, check language (DSL), checks, lineage, attribution, verification, experiment, lifecycle, evidence and redaction.**
- **Refund Desk: five steps, seven checks, repair rules, the stale-price fault, and an independent retail store used as the source of truth.**

Figure 5.2 shows these components and the interfaces between them.

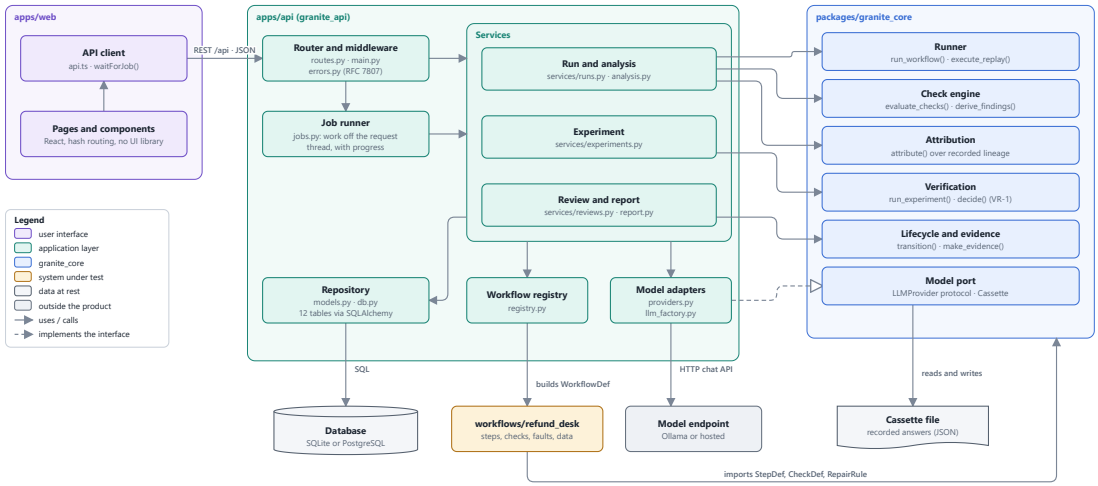


Figure 5.2: Component diagram

5.3 Interfaces and Dependencies

HTTP API. Every endpoint sits under `/api` and is documented automatically at `/docs`.

Table 5.2: Interfaces and Dependencies

Method and path	Purpose	Main responses
GET `/workflows`, `/workflows/{key}/examples`	Registered workflows and example tasks	200
POST `/runs`	Start a run (task, fault, seed, mode)	202 with a job id; 422 invalid input; 429 rate limit
GET `/runs`, `/runs/{id}`, `/runs/{id}/graph`	List, detail (trace, checks, findings), data-flow graph	200; 404
GET `/findings`, `/findings/summary`, `/findings/{id}`	List with filters, counts by status, detail with allowed actions	200; 404
GET `/findings/{id}/report?format=md|json`	Export the investigation	200
POST `/hypotheses/{id}/experiments`	Start a verification (N, M)	202; 409 not allowed now
GET `/experiments/{id}`	Replays, summary and verdict	200
POST `/findings/{id}/reviews`	Record a decision	201; 409 illegal transition; 422 missing comment
GET `/evidence/{id}`	One evidence record with hash check	200
GET `/jobs/{id}`	Job status and progress	200
POST `/settings/test-llm`	Check the model connection	200

Internal contracts. The core exposes:

- `run_workflow` **and** `execute_replay` **(runner);**
- `evaluate_checks` **and** `derive_findings` **(checks);**
- `attribute` **(attribution);**
- `plan_replay`, `decide` **and** `wilson` **(verification);**
- `run_experiment` **(experiment);**
- `transition` **(lifecycle);**
- `make_evidence` **and** `verify_evidence` **(evidence).**

A `WorkflowDef` describes a workflow: its steps with declared reads, its checks, repair rules and truth functions. Traces follow the published Granite Trace v1 JSON Schema.

External dependencies: Ollama with llama3.2:3b (live mode), SQLite or PostgreSQL, Render for hosting, and GitHub Actions with the Container Registry for CI/CD.

5.4 Architecture Decisions and Rationale

Table 5.3: Architecture Decisions and Rationale

ADR	Decision	Rationale	Alternatives rejected
001	Modular monolith with one dependency direction	Simple for a three-person team to run, test and deploy; imports enforce the boundaries	Microservices (too much operational cost)
002	Build our own workflow runner first	Gives full control of the trace, and of replay from any step	Instrumenting CrewAI or LangGraph first (no replay control)
003	SQLAlchemy persistence; one JSON trace document per run	One source for hashing and replay; fewer tables to keep consistent	Separate step and edge tables
004	Check language on Python's syntax tree, with a whitelist	Safe, explainable code we can defend	`eval` (unsafe); a third-party evaluator
005	Provenance-first attribution	Deterministic, cheap and testable; an LLM opinion is only a labelled second opinion	LLM-judge attribution as the core
006	Verdict rule VR-1, with controls	Separates real causes from model randomness; thresholds are versioned	A single re-run; unversioned thresholds
007	Free hosting on Render	No cost; deploys from Docker	Paid hosting; Hugging Face Spaces (its free tier changed)
008	Provider abstraction and recorded mode	Demos and tests run offline and repeatably	Calling a model on every run

ADR	Decision	Rationale	Alternatives rejected
009	Granite Trace v1 schema	A stable input contract for import and the SDK	Framework-specific formats
010	Short-lived branches merged into `main` through pull requests	Changes are reviewed and CI-checked; simple for a small team	Git-flow with a `develop` branch (revised 11 Oct)

5.5 Design Evolution

Table 5.4: Design Evolution

Aspect	At the proposal	Now (Mid-1)	Why it changed
Product shape	Ten mechanisms; a read-only PoC console	One workflow and eight user-facing features, built around the verified finding	Panel comments A1, A2, A5
Detection	Comparison against an answer key (Spike 4)	Executable checks against an independent source of truth	No answer key exists in production; this is the gap ErrorProbe names
Attribution	An ErrorProbe-style LLM pipeline (Strategist, Investigator, Arbiter)	Deterministic provenance; an LLM opinion is optional	Spike 1 reached only 0–25% step accuracy with a 3B model; deterministic code is testable
Verification	"Replay the fix" in a single run	N fix replays plus M controls, rule VR-1, a Wilson interval, and levels V0–V3	Panel comment A4; the model is not deterministic
Evidence	Not defined	Typed, SHA-256-hashed, append-only records, plus review and audit records	Panel comment A4
Persistence and hosting	PostgreSQL (Neon), Hugging Face Spaces	SQLite (PostgreSQL supported), Render	Simplicity; the free tier changed
Baseline	Reproduce ErrorProbe within $\pm$10 points	Last-agent and LLM-judge baselines; ErrorProbe-lite later	Feasibility (change record C-01)

Aspect	At the proposal	Now (Mid-1)	Why it changed
Branching	`main` plus `develop`	Short-lived branches into `main`	Team size (ADR-010 revision, C-03b)
Diagrams	Hand-drawn	Generated from the code and checked in CI	Diagrams must match the code (C-03a)

5.6 Domain Model

5.7 Domain Model

A Workflow (WorkflowDef) has ordered Steps (StepDef). Each step has an agent name, a kind (llm, tool, code or handoff), and the earlier fields it reads. The workflow also owns its Checks (CheckDef) and Repair rules.

Running a workflow produces a Run. Its Trace holds one recorded Step per executed step, with that step's inputs, outputs and timing. Evaluating the checks produces Check results: pass, fail, abstain or error.

A failed outcome check opens a Finding, with a severity and a symptom step. Attribution then produces ranked Hypotheses, each naming a step. Verifying a hypothesis creates an Experiment, made of Replays, which ends in a Verdict. A Review records the human decision on the finding.

Every artefact along the way is stored as an Evidence record with a hash, and every state change is written to the Audit log. A Fault injection record holds the ground truth of an injected fault, and attribution never reads it.

5.8 Data Design

The database has twelve tables. A data dictionary of 120 columns is generated from the models.

- `projects` **and** `runs` **hold the workflow and each execution. The trace is stored as one JSON document with its SHA-256 hash.**
- `check_results` **holds each check's status, with observed and expected values.**
- `fault_injections` **holds the ground truth of an injected fault.**

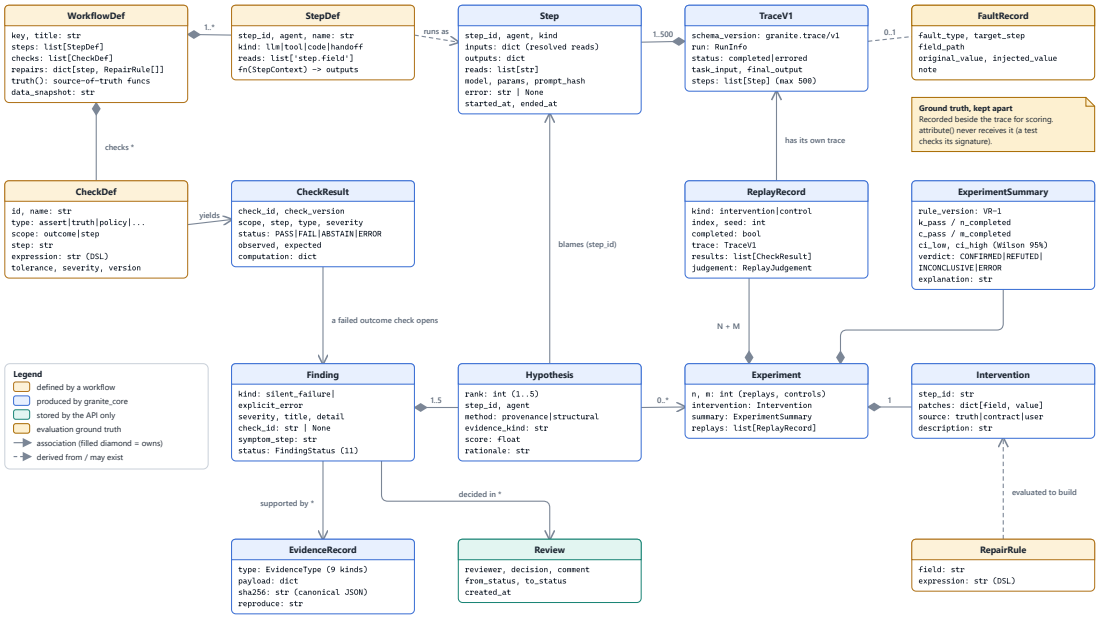


Figure 5.3: Domain model (class diagram) generated from the code

- **`findings` holds each detected failure, with its severity, symptom step and lifecycle status.**
- **`hypotheses` holds the ranked suspects, with method, evidence class and score.**
- **`experiments` and `replays` hold verification runs and the outcome of each replay.**
- **`reviews` holds the human decisions.**
- **`evidence` holds typed, hashed, append-only artefacts.**
- **`audit_log` records every state change.**
- **`jobs` tracks background work.**

Evidence and audit rows have no update path, so the record cannot be rewritten. The trace itself follows the Granite Trace v1 schema: a RunInfo plus a list of Steps.

5.9 Process / Behavior Design

Finding lifecycle. A finding starts as *open*. It becomes *hypothesized* once hypotheses are ranked, and *verifying* when an experiment starts. The verdict then moves it to

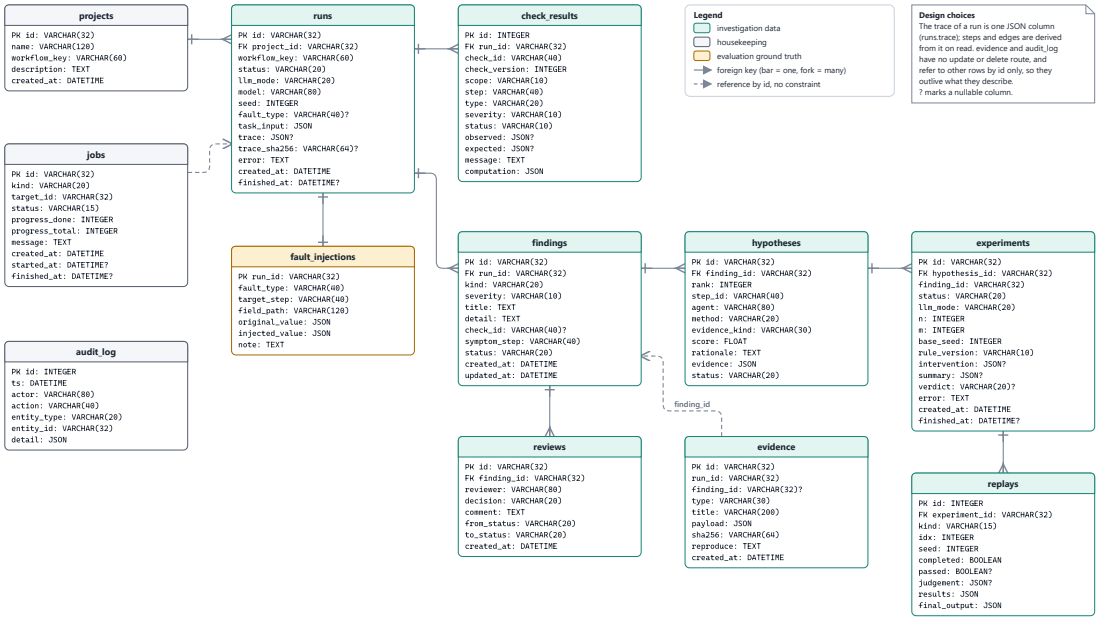


Figure 5.4: Entity relationship diagram of the twelve tables

***confirmed*, *refuted* or *inconclusive*. If the experiment errors, it returns to hypothesized.**

A refuted or inconclusive finding can be verified again, either on the next hypothesis or with more replays. A confirmed or inconclusive finding is reviewed, which makes it *accepted* or *rejected*. An accepted finding is then *resolved*, or closed as *won't fix*. Rejected, resolved and won't-fix are final. Any other event is refused as an illegal transition.

Verification levels follow the state: V0 detected, V1 evidence-reproduced, V2 replay-verified, V3 human-confirmed.

Verification sequence. When the user presses Verify:

1. **The API checks that a repair rule exists and stores a queued experiment.**
2. **A background job builds the intervention, which is the corrected value.**
3. **It plans the replay: which steps keep their recorded outputs, and which re-run.**
4. **It runs N replays with the correction and M without it.**
5. **It judges each replay against the target checks and applies VR-1.**
6. **It stores the verdict and the replay outputs as evidence, and moves the finding.**

Data flow. At level 0, the Developer and Reviewer send tasks, verification requests and decisions to Granite. Granite exchanges prompts and answers with the LLM Provider, and returns runs, findings, verdicts and reports. At level 1, the processes are run workflow, evaluate checks, attribute, verify and review. These connect to the stores shown in the ER diagram.

Figure 5.5 shows the lifecycle, Figure 5.6 the activity flow across the five actors, and Figures 5.7 and 5.8 the data flow at levels 0 and 1.

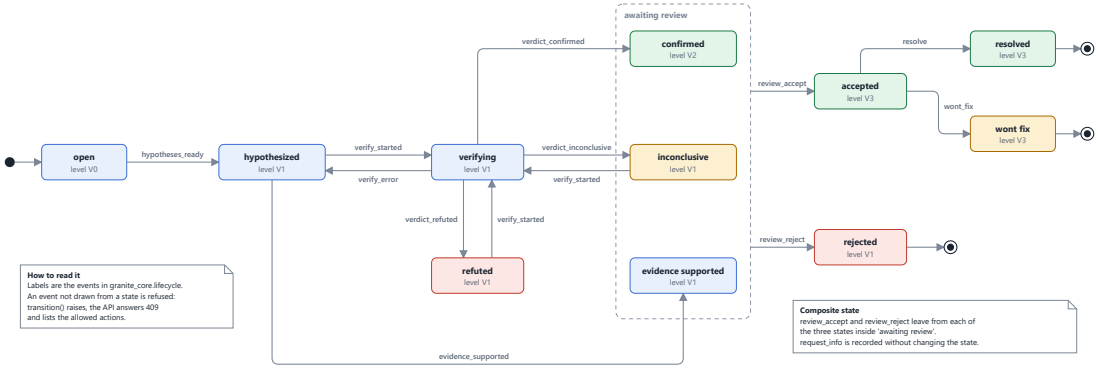


Figure 5.5: Finding state machine, read from the lifecycle code

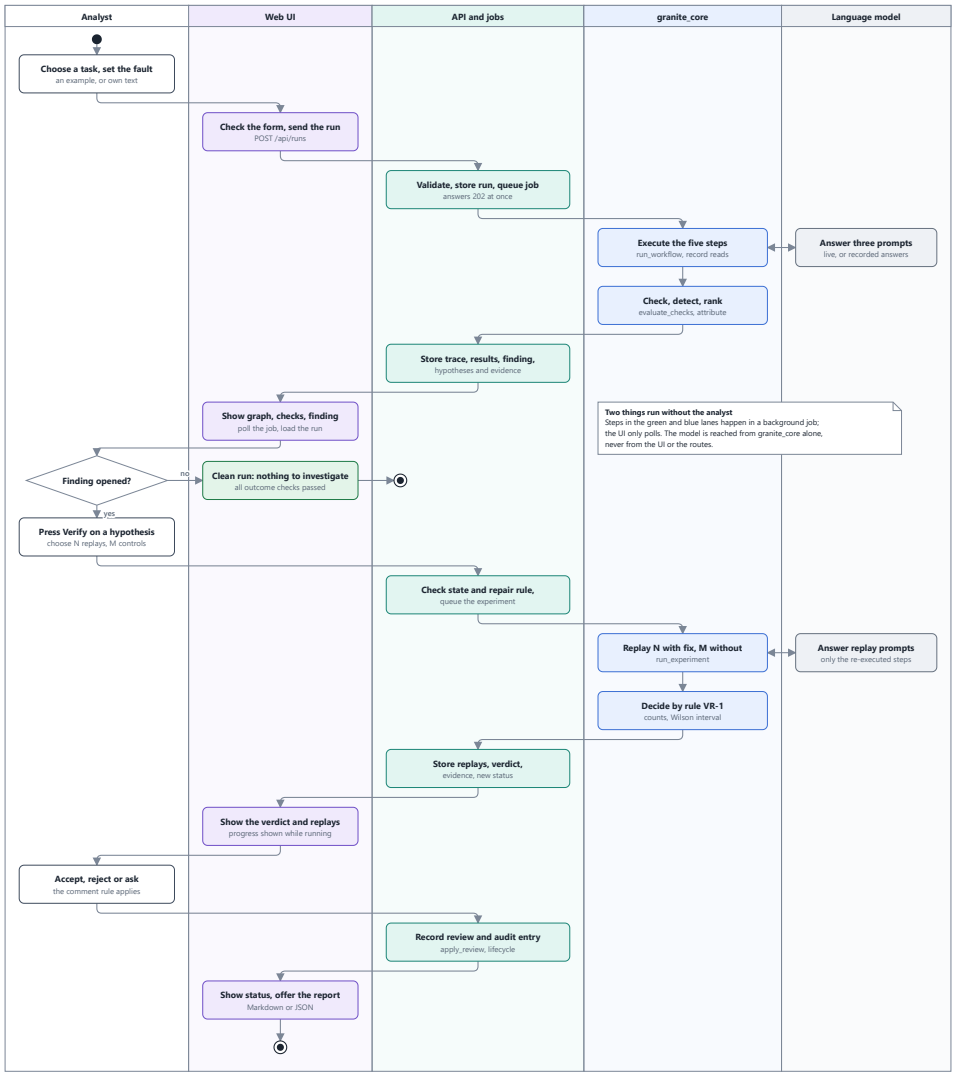


Figure 5.6: Activity diagram of an investigation



Figure 5.7: Data flow diagram, level 0 (context)

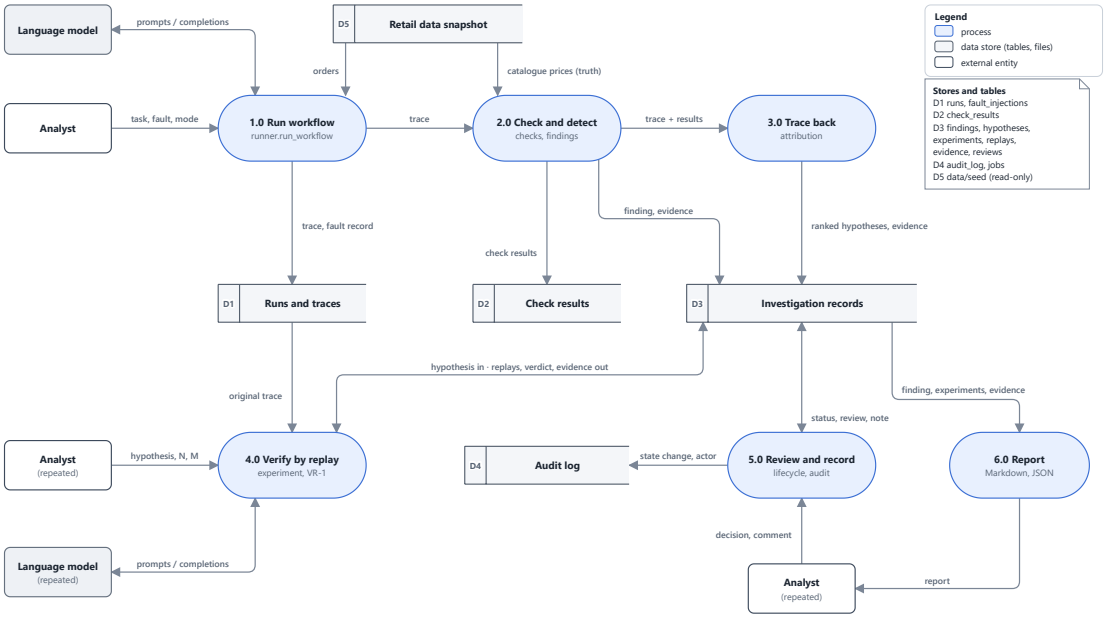


Figure 5.8: Data flow diagram, level 1

Chapter 6

Iteration 1: Design and Implementation

This chapter covers the first iteration. It gives the objectives and the use cases chosen, then the design of the Refund Desk workflow, which is the system under test. It then explains how the implementation is organised, the algorithms at its core, the tools used, and the evidence that the iteration works.

6.1 Iteration Objectives

Build one complete, working investigation slice. It must:

- **run a real multi-agent workflow, on real data, with a real model;**
- **detect a silent failure without an answer key;**
- **trace that failure to its origin;**
- **verify the cause using controls;**
- **record a human decision;**
- **export the evidence.**

It must ship with automated tests, CI, a deployed instance, and design artefacts that match the code.

6.2 Selected Use Cases / Features

We chose UC-03 (run), UC-05 (inspect), UC-06 (analyse), UC-07 (investigate), UC-08 (verify), UC-09 (review) and UC-12 (history), plus the report part of UC-11.

These became eight features: F1 Run, F2 Check and detect, F3 Trace back, F4 Verify, F5 Review, F6 Evidence and report, F7 Find, and F8 Guide.

6.3 Design

The Refund Desk has five steps. Each one declares what it reads.

1. **s1 Intake (LLM) pulls the order number and items out of the customer's message.**
2. **s2 Order Lookup (tool) fetches the order and its prices from the store.**
3. **s3 Policy (plain code) decides eligibility from the order's status, owner and items.**
4. **s4 Refund Calculator (LLM) adds up the refund.**
5. **s5 Writer (LLM) writes the reply.**

Seven checks are attached to these steps:

- **two outcome checks: the stated refund equals the catalogue total of the refunded items, and the decision follows policy;**
- **five step contracts, one per step.**

Repair rules say how to work out the correct output of a step. Verification uses these as its intervention. The stale-price fault lowers one requested item's price by $0.25 at s2, and raises no error while doing it. See Figure 1.1.

6.4 Implementation

Table 6.1: Implementation

Package	Files	Tests	Responsibility
`packages/granite_core`	13	207	Trace, runner, check language, checks, lineage, attribution, verification, experiment, lifecycle, evidence, LLM provider interfaces, redaction
`workflows/refund_desk`	5	49	Steps, checks, repair rules, fault, retail store
`apps/api`	20	71	Endpoints, schemas, models, services, jobs, providers, errors
`apps/web`	16	32	Five screens and shared components
`tests/system`	—	22	Black-box tests over HTTP

A run request flows like this:

1. **The New run page posts the task.**
2. **The route checks the token and the rate limit, then returns 202 with a job.**
3. **The run service validates the task, stores a queued run and starts the job.**
4. **The runner executes the five steps and records the trace, applying the fault if one was requested.**
5. **The analysis service stores the trace and its hash, evaluates the checks, opens findings, ranks hypotheses and writes evidence.**

A verification request flows through the experiment service, then `build_intervention` and `plan_replay`, then `run_experiment` (N + M replays, each with its own seed), then `decide` (VR-1) and `transition` (lifecycle).

Figure 6.1 shows the modules and functions involved.

6.5 Completed Features / Module

All eight features are built and tested:

- **F1: run a workflow and record every step.**

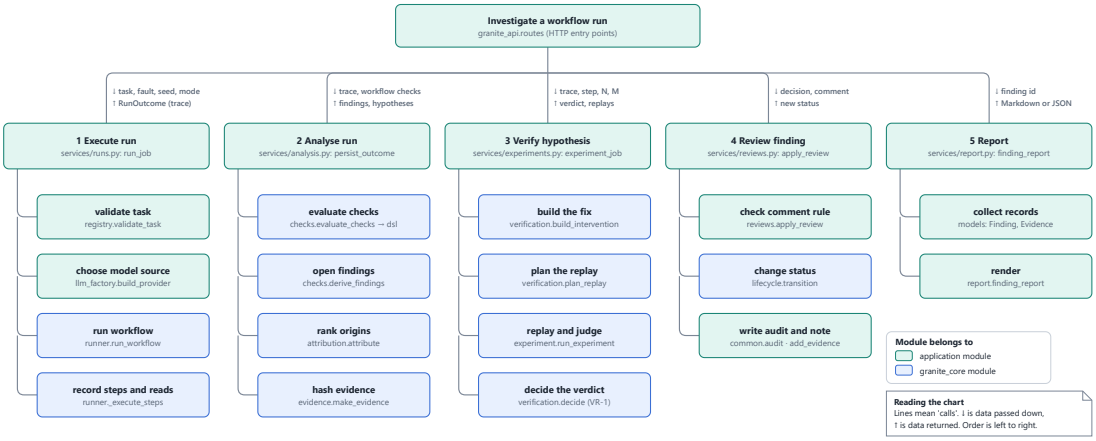


Figure 6.1: Structure chart with the real function names

- **F2: check the outcome and detect silent failures.**
- **F3: trace back to where the failure began.**
- **F4: verify by controlled replay.**
- **F5: review.**
- **F6: evidence and report export.**
- **F7: find earlier runs and findings.**
- **F8: the in-product guide and API reference.**

These are not built, and we do not claim them: trace import, a second workflow, a check editor, fix suggestions, PDF export, and redaction of stored data.

6.6 Technical Implementation Details

Reproducible setup. These steps come from the setup manual. They work on Windows, Linux and macOS with Python 3.11 and Node 20 or later.

```
python -m venv .venv && source .venv/bin/activate # Windows: .venv\Scripts\
    activate
pip install -e "packages/granite_core[dev]" -e "workflows/refund_desk[dev]" -e "
    apps/api[dev]"
(cd apps/web && npm ci && npm run build)
cd apps/api
GRANITE_LLM_MODE=recorded GRANITE_STATIC_DIR=../web/dist uvicorn granite_api.main
    :app_from_env --factory --port 8000
```

Then open `http://127.0.0.1:8000`. Recorded mode needs no model and no key. For live mode, install Ollama, run `ollama pull llama3.2:3b` and set `GRANITE_LLM_MODE=live`. The container runs with `docker compose up`. We repeated this setup on a second team member's laptop on 11 October, and all 381 tests passed.

Key technical details:

- **Each step's inputs come from its declared reads. So the data flow is recorded, not guessed.**
- **Traces are stored as canonical JSON and hashed with SHA-256.**
- **Recorded answers are keyed by a hash of the model, messages, parameters and seed. A missing answer raises an error rather than inventing one.**
- **Model outputs are requested as JSON and parsed. Invalid JSON fails the step visibly.**
- **Background work runs in a thread pool with a job table.**
- **Errors follow RFC 7807. Internal exception text never reaches the client.**

6.7 Algorithms and Technical Decisions

Check language. A check is an expression, such as:

```
approx(number_in(s5.message), truth.catalog_total(pluck(s3.items_
to_refund, 'item_id')))
```

Python's `ast` module parses it. It is accepted only if every syntax node and function is on a whitelist, and it stays within 500 characters, 150 nodes and a depth of 14. Our own interpreter then evaluates it. We never use `eval`. A missing field gives ABSTAIN, not PASS.

Attribution. The algorithm starts at the symptom step and walks backwards along the lineage edges that carry the failing field. It classifies each step by its evidence:

- **truth mismatch: weight 1.0;**
- **contract failure: 0.8;**
- **hand-off drift, where a step's input differs from what the step before produced: 0.7;**
- **no check defined: 0.2;**

- **consistent: 0.**

Candidates are ranked by weight, then by earliest position, with a decay for distance from the symptom. If no step has direct evidence (weight $\geq$ 0.7), every candidate is labelled "structural only, needs verification". Complexity is O(V + E). No language model is involved.

Verdict rule VR-1. Let k of N intervention replays pass, so p = k/N, and c of M controls pass, so q = c/M. Then:

- **CONFIRMED if p $\geq$ 0.8 and q $\leq$ 0.2;**
- **REFUTED if p $\leq$ 0.2;**
- **INCONCLUSIVE otherwise;**
- **ERROR if fewer than N replays complete.**

We report a 95% Wilson interval, but at N = 5 it does not decide the verdict. A replay that causes a new check failure counts as not passing. This is the regression guard. The thresholds are versioned, so changing them would create VR-2.

Replay plan. To test a hypothesis about step s:

- **keep the outputs of every step before s from the original trace;**
- **replace s's output with the corrected value from its repair rule;**
- **re-run only the steps after s.**

Frozen steps therefore cost no model calls.

Figures 6.2 to 6.5 show the four algorithms.

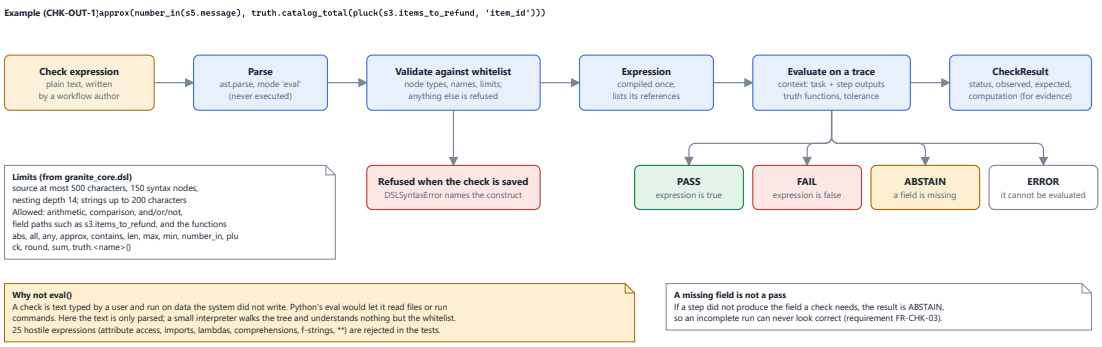


Figure 6.2: Check language: from expression text to result

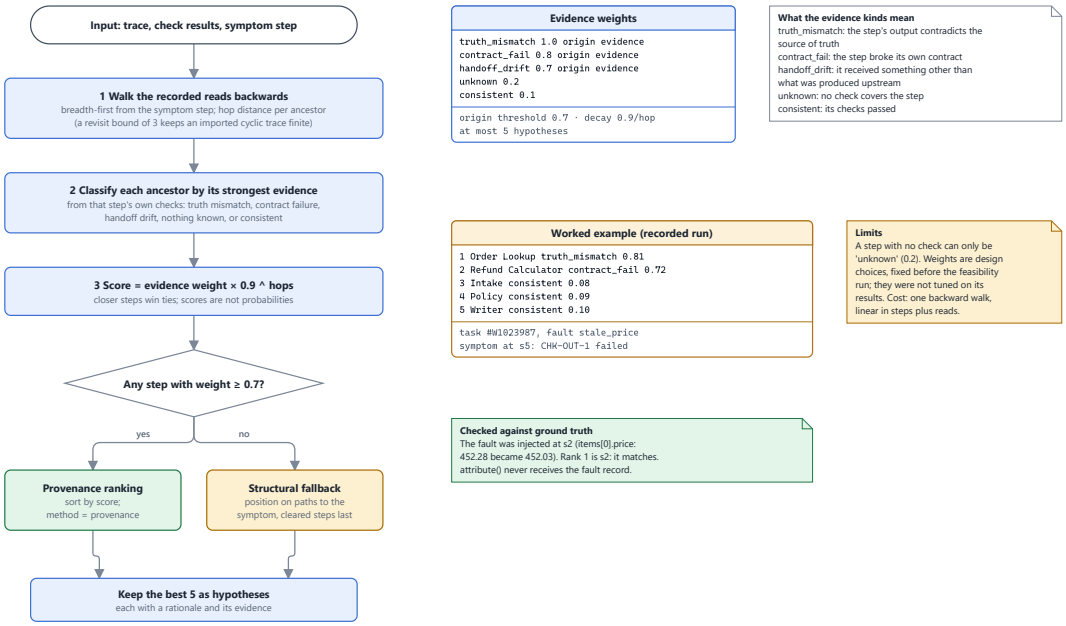


Figure 6.3: Attribution: ranking where a failure began

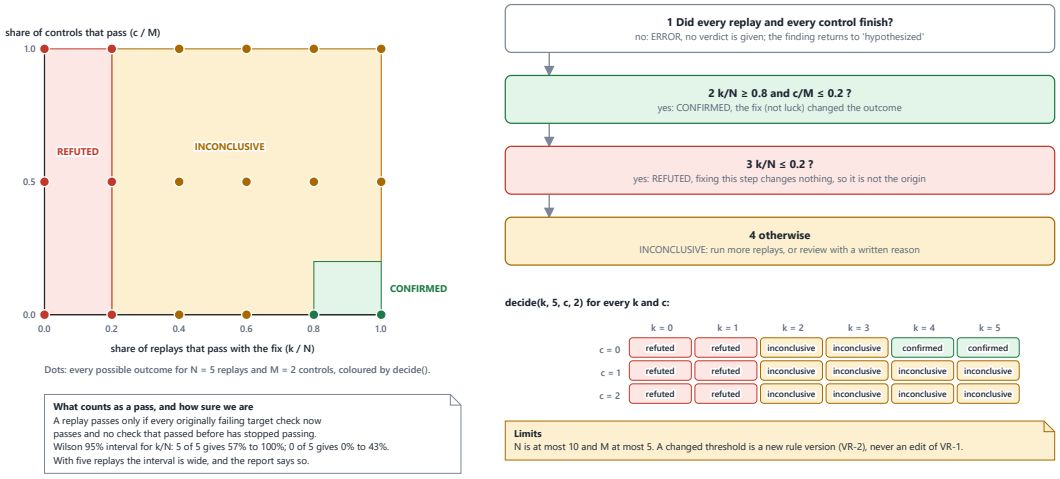


Figure 6.4: Verdict rule VR-1

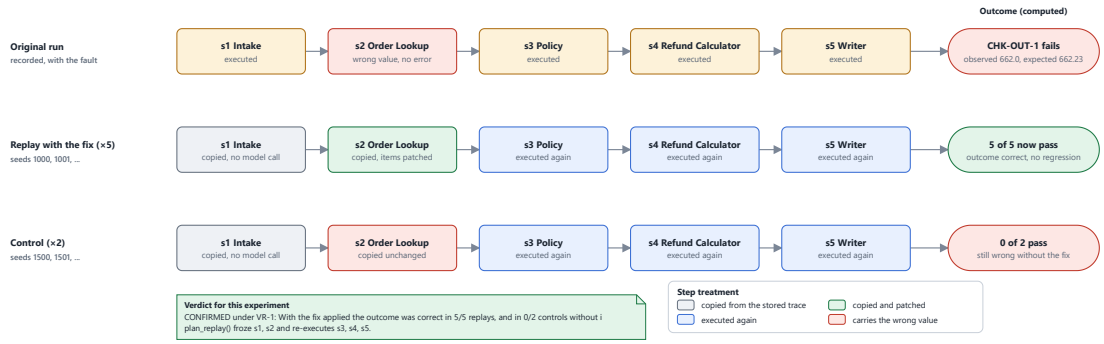


Figure 6.5: Replay plan: correct one step, keep everything before it

6.8 Tools, Frameworks and Technologies

- **Backend: Python 3.11, FastAPI, Pydantic, SQLAlchemy, uvicorn.**
- **Frontend: React 19, TypeScript, Vite.**
- **Data: SQLite, with PostgreSQL supported.**
- **Model: Ollama with llama3.2:3b.**
- **Testing: pytest, Hypothesis, Vitest, Testing Library, axe-core.**
- **Static analysis: ruff, mypy (strict), ESLint, bandit, pip-audit, npm audit, gitleaks.**
- **Delivery: Docker and Docker Compose, GitHub Actions, GitHub Container Registry, Render.**

We used AI-assisted development (Claude Code). Section 8.7 discloses this.

6.9 Demonstration Evidence

The product is deployed at `https://granite-demo.onrender.com`, and it runs locally from the setup steps above. Screenshots of every screen, the error state and the phone layout are in `evidence/demos/`.

The demonstration script, with its expected values, is `docs/manuals/demo-runbook.md`:

1. **Run the first example with the stale-price fault.**

2. **The run completes, but the Order Lookup and Writer steps fail their checks.**

3. **Open the finding. Order Lookup is ranked first.**

4. **Verify it: CONFIRMED, 5/5 fixed replays, 0/2 controls.**

5. **Verify the calculator: REFUTED.**

6. **Accept with a comment, then download the report.**

7. **Show that a clean run raises no finding, and that invalid input and an unreachable model are both handled.**

Figures 6.6 to 6.11 show the product during the demonstration.

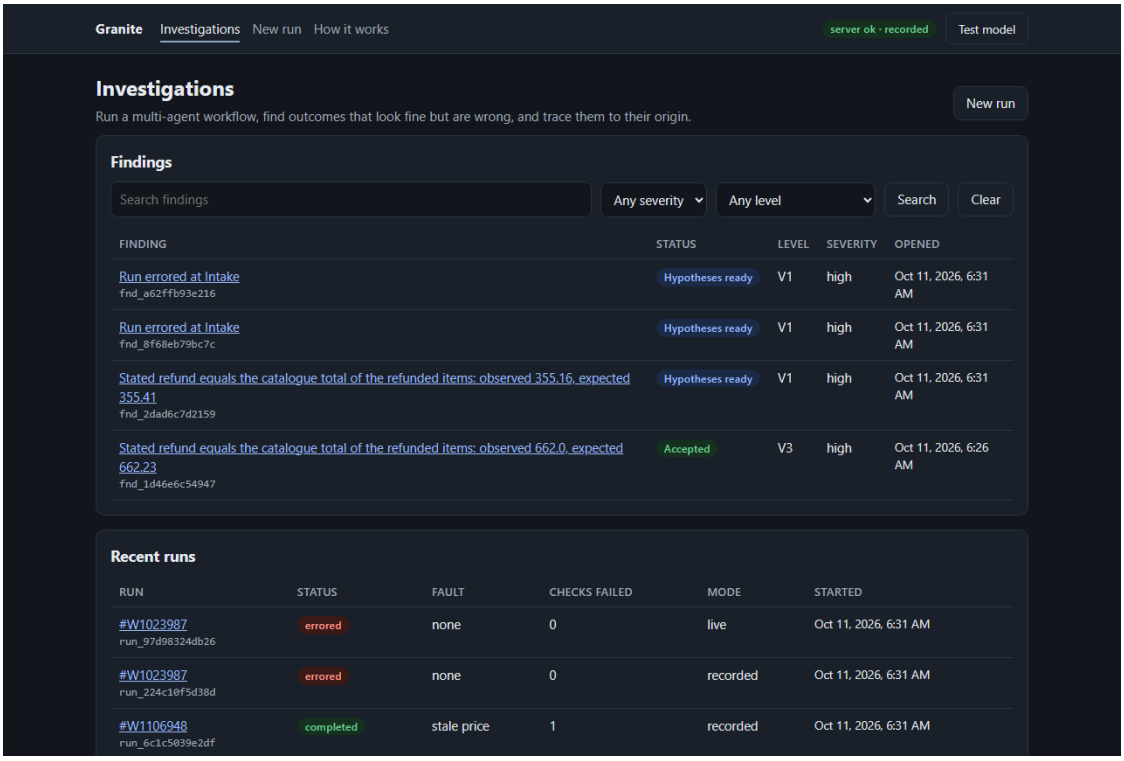


Figure 6.6: Investigations list: earlier runs and findings, with filters and search

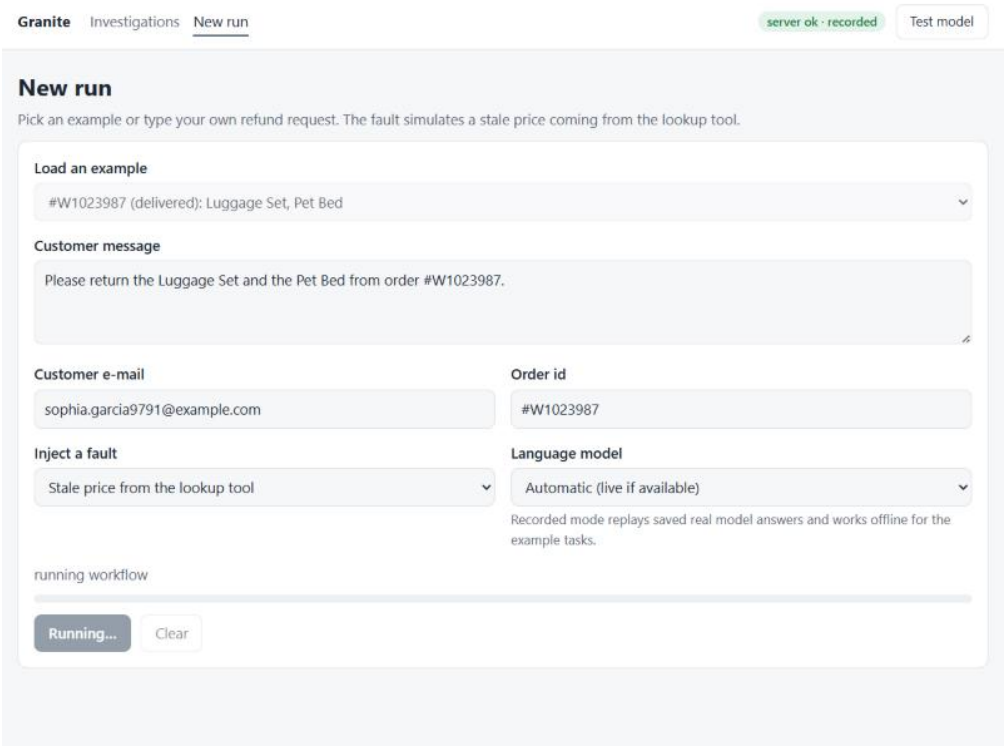


Figure 6.7: New run: validated input form

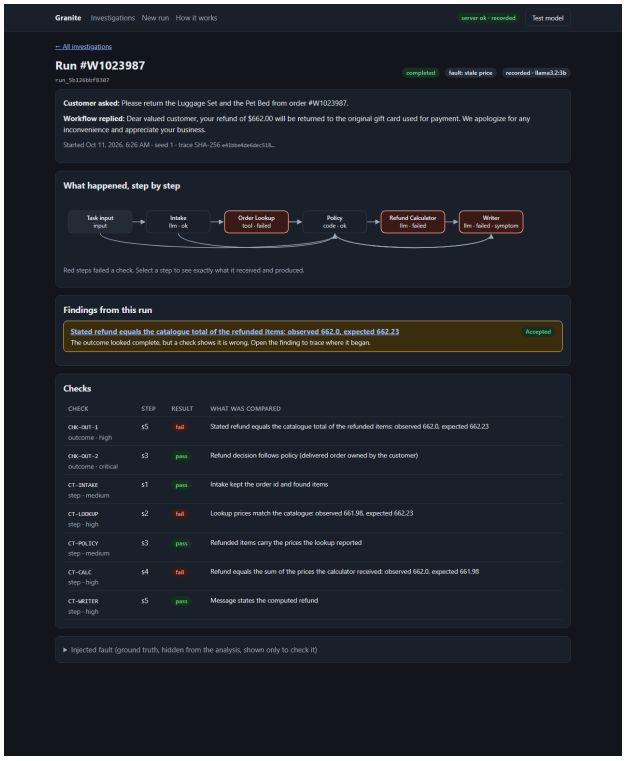


Figure 6.8: Run page: pipeline graph with failed steps and the seven checks

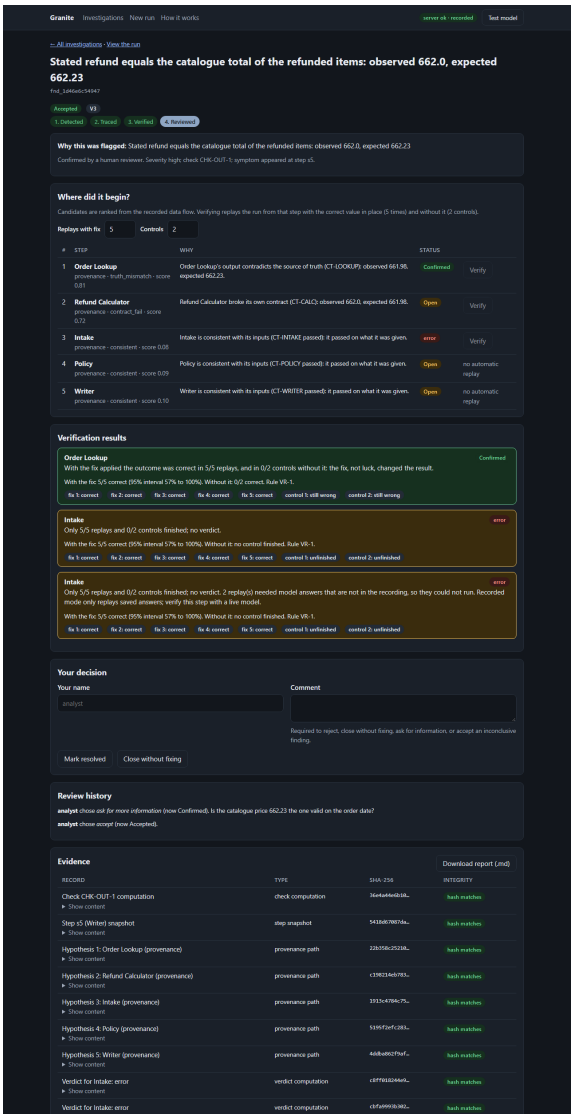


Figure 6.9: Finding page: ranked hypotheses, verification results, review and evidence

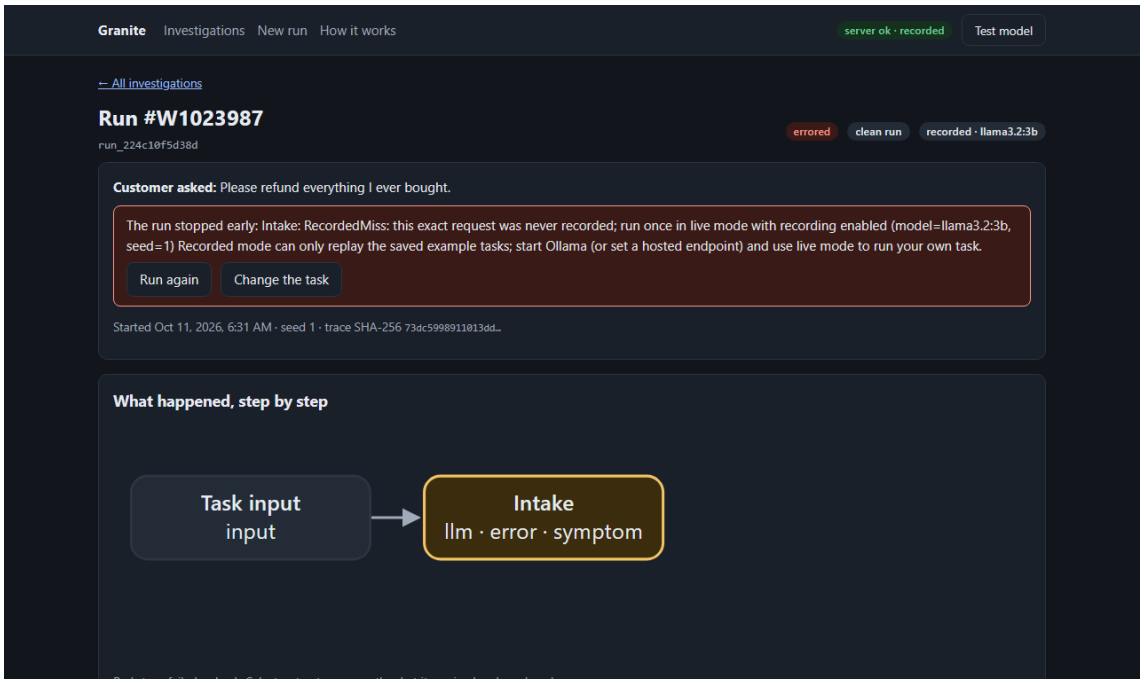


Figure 6.10: Failure case: a run that cannot be answered ends as errored with the reason



Figure 6.11: Guide page: the eight features with their inputs and outputs (feature F8)

6.10 Iteration Outcome

Iteration 1 met its objective. The full investigation loop works end to end, on real data with a real model. It is deployed, covered by 381 automated tests and guarded by CI. The pre-registered feasibility run met every threshold.

Four shortfalls carry into Iteration 2:

- **redaction before storage;**
- **a usability test with people outside the team;**
- **fully automated deployment;**
- **visible per-member ownership of the code.**

Chapter 7

Testing and Early Validation

This chapter covers how Granite was tested and validated. It gives the strategy, the test levels and techniques, the test cases and their results. It then covers the pre-registered feasibility validation, the edge and failure cases, the defects found and fixed, and what the current evidence does and does not support.

7.1 Testing Strategy

We keep three questions apart:

- **Testing asks whether Granite behaves as specified.**
- **Evaluation asks whether it is accurate compared with baselines.**
- **Acceptance asks whether users can and will use it.**

At Mid-1, testing is complete for the slice we built. Evaluation is a pre-registered feasibility run. Acceptance has a protocol ready, but has not been run.

Every requirement has at least one test that can fail. Failure and edge cases are tested at every level. All suites run in CI on every push and pull request, behind a 90% coverage gate.

7.2 Test Levels and Techniques

Table 7.1: Test Levels and Techniques

Level	Scope	Tools	Count
Unit and property	Core library: check language, checks, attribution, verdict rule, lifecycle, evidence, redaction	pytest, Hypothesis	207
Unit and integration	Refund Desk steps, store, fault, verification	pytest	49
Integration	API with database, runner with recorded model, job runner	pytest, HTTP test client	71
System (black box)	User journeys and failure modes over real HTTP, against a running server	pytest	22
UI component and accessibility	Screens, components, axe-core checks in light and dark themes	Vitest, Testing Library, axe-core	32
Static and security analysis	Lint, strict types, security, dependency vulnerabilities, secrets	ruff, mypy, ESLint, tsc, bandit, pip-audit, npm audit, gitleaks	every push

We use these techniques:

- **Equivalence partitioning: check statuses, fault types and classes of valid and invalid input.**
- **Boundary values: the verdict thresholds at 0.2 and 0.8, and the input length limits.**
- **A decision table for VR-1, with 20 rows.**
- **State-transition testing: every legal transition and several illegal ones.**
- **Property-based testing: the check language never crashes on arbitrary text, and never accepts forbidden syntax.**
- **Error guessing: model output that is not JSON, an unreachable model, an unrecorded task.**
- **Negative tests: a clean run must produce no findings.**

7.3 Test Cases

The 22 system test cases fall into three groups.

Core journeys:

- **SYS-CORE-01: a full investigation of a silent failure.**
- **02: a clean run raises no alarm.**
- **03: refute a wrong hypothesis, then confirm the right one.**
- **04: the run graph marks the failing steps.**
- **05: runs and findings can be found again.**
- **06: the running system documents its own interface.**
- **07: the same server serves the web interface.**

Errors:

- **SYS-ERR-01: invalid input is refused, one message per field.**
- **02: requests for items that do not exist.**
- **03: the lifecycle cannot be skipped.**
- **04: recorded mode cannot answer an unrecorded task.**
- **05: a hypothesis that cannot be tested is refused, not guessed.**
- **06: limits are enforced.**
- **07: the model is down.**

Security and reliability:

- **SYS-SEC-01: security headers on every response.**
- **02: evidence is append-only.**
- **03: ground truth stays out of evidence.**
- **04: with a token configured, only the health endpoint is public.**
- **05: too many requests are throttled.**

- **SYS-REL-01: a restart loses nothing.**
- **02: concurrent work is not lost.**
- **03: recorded runs are reproducible.**

Each case names its requirement, its steps and the expected result. See Appendix C.

7.4 Test Results

On 11 October all 381 automated tests passed:

- **core library: 207 tests, 97.5% line coverage;**
- **Refund Desk: 49 tests, 95.4%;**
- **API: 71 tests, 96.8%;**
- **system: 22 tests;**
- **web: 32 tests.**

Static analysis reported nothing. CI passed on GitHub for the latest commits, including the container build and a smoke test inside the container.

Against the public deployment, the smoke test passed. The system suite gave 18 passed and 4 skipped by design, because those four cases start their own server. We also ran the suites on a second machine. Repeating them there exposed one intermittent defect, D-26, described below.

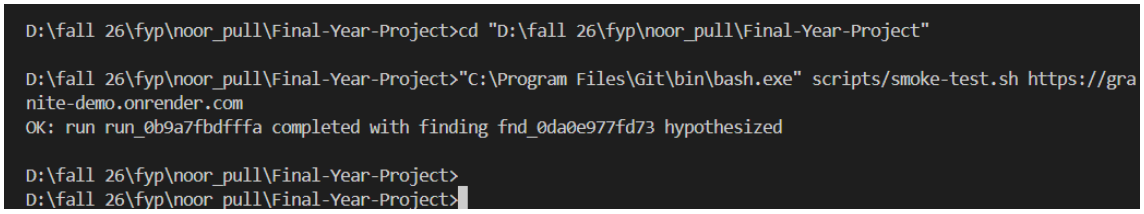


Figure 7.1: Smoke test passing against the public deployment

7.5 Acceptance Testing / Validation

We pre-registered a feasibility validation before running it. The design: ten tasks, each run clean and again with the stale-price fault, using the local model llama3.2:3b, with N = 5 and M = 2. The thresholds were fixed in advance.

Table 7.2: Feasibility validation results

Measure	Threshold	Result
Fault detected in faulty runs	**$\geq$ 90%**	**10/10**
False alarms on clean runs	**0**	**0**
True origin ranked first	**$\geq$ 80%**	**10/10**
True origin CONFIRMED	**$\geq$ 80%**	**9/10**
Innocent step REFUTED	**$\geq$ 80%**	**10/10**
Runs completed	**$\geq$ 95%**	**20/20**

Every threshold was met. One clean run did open a finding, but it was a real error, not a false alarm: the model told a customer 4,047.67 instead of 3,047.67. Nobody injected that, and Granite caught it.

We state the limits plainly:

- **Ten tasks, one fault type, one small model and synthetic data. So the intervals are wide: the lower bound for M4 is 60%.**
- **Four of the tasks were not blind.**
- **Attribution is easy here, because the origin step has its own check.**
- **The refutation test is weak.**

User acceptance testing has a protocol ready: five tasks, the SUS questionnaire and thresholds fixed in advance. It will run with three outside participants before FYP-1 Final.

7.6 Edge and Failure Cases

These are tested, and shown in the demonstration:

- **A clean run produces no finding.**
- **Invalid input is refused, with one message per field.**
- **An unreachable model or an unrecorded task ends the run as "errored", with a reason and a retry button.**
- **Verifying an innocent step gives REFUTED.**
- **A step with no repair rule cannot be verified, and the system says why.**

- **Accepting before verification is refused, and so is rejecting without a comment.**
- **Hostile check expressions are rejected.**
- **Tampered evidence is detected.**
- **Too many requests are throttled.**
- **A restart marks interrupted work as failed instead of losing it.**

The feasibility run also exposed a real design limit. When two faults sit in one chain, correcting a single step cannot produce a CONFIRMED verdict (D-19).

7.7 Defects and Corrective Actions

We have logged twenty-six defects. Fixed ones include:

- **D-01: the stale-price fault hit the first item instead of the requested one. Fixed, with a regression test.**
- **D-15: a non-ASCII token caused a server error instead of a 401.**
- **D-16: oversized request bodies were read into memory. They are now refused with a 413.**
- **D-22: the health check was slow on machines with no model. It is now cached.**
- **D-23: a verification that reached no verdict gave no reason.**
- **D-24: validator wording was shown to users.**
- **D-26: a review decision was answered before it was saved, so an immediate next request could see the old state. Repeating the system tests on a second machine found it. We fixed it by committing before responding, and the test then passed 15 runs out of 15.**

Open defects, each with a plan: the redaction gap (D-07), untested PostgreSQL (D-08), recorded answers tied to exact prompts (D-10), the two-fault limit (D-19), and rank and score disagreeing for low-ranked steps (D-25).

7.8 Current Validation Status

The slice we built is fully tested at every level and works on a live deployment. The feasibility run supports one claim: the run, check, trace-back and verify loop works end to end, on a real small model and public data.

It does not yet support claims about accuracy in general, about harder faults, about other workflows, or about usability with real users. Those are the targets of the Iteration 2 benchmark, which is at least 120 runs with baselines, and the usability pilot.

Chapter 8

Project Management and Development Evidence

This chapter reports how the project was managed, with the evidence behind it: work completed and remaining, individual responsibilities and contributions, repository and version-control practice, milestones, tools and AI-assisted development, external components, risks, security and privacy, and current limitations.

8.1 Work Completed

Granite's first iteration, the core investigation slice, was built between 6 and 11 October 2026 and released as version 0.1.0.

Granite does one job. When a workflow of several AI agents gives an answer that looks right but is wrong, Granite finds the step where the error began and shows the proof. The product follows a fixed sequence: Run, Check, Detect, Trace back, Verify, Review and Record. Every stage in that sequence is a working feature.

A user can run the Refund Desk workflow, where five agents handle a customer's refund request on real retail orders. Granite records each step's inputs and outputs as a hashed trace. Before the run, a fault can be injected, such as a stale price from the order lookup.

After every run, seven executable checks are evaluated against a source of truth that the workflow does not control. Two of them judge the final outcome. The other five act as contracts for individual steps. If an outcome check fails on a run that completed without any error, Granite opens a finding for a silent failure.

Granite then traces the wrong value back through the data flow between steps, and ranks the steps where it most likely entered. No language model is involved. The

top hypothesis can then be verified by controlled replay: the workflow is re-run five times with the suspected step corrected, and twice without the correction as controls. A fixed rule, VR-1, returns CONFIRMED, REFUTED or INCONCLUSIVE.

A person then accepts or rejects the finding. Every decision is stored as append-only evidence with a SHA-256 hash, and the investigation can be exported as a Markdown or JSON report. Earlier runs and findings can be searched and filtered. The product explains itself through a guide page and an API reference.

Alongside the product, the team completed:

- **a re-check of the master plan against the Mid-1 rubric;**
- **a reorganised repository, with automated structure and link checks;**
- **continuous integration on GitHub Actions, first green run on 9 October;**
- **a design pack of 21 diagrams, generated from the code;**
- **a suite of 22 system-level tests;**
- **setup and tool manuals, and the panel action register.**

On 11 October the product was deployed publicly on Render at `https://granite-demo.onrender.com`, then verified with a smoke test and the system test suite.

All 381 automated tests pass. Table 8.1 shows how they are distributed. Lint, strict type checking, security scanning and dependency audits report nothing.

Table 8.1: Automated test results, 11 October 2026

Suite (level)	Tests	Line coverage
Core library (unit, property-based)	207	97.5%
Refund Desk workflow (unit, integration)	49	95.4%
API (integration)	71	96.8%
System (black box over HTTP)	22	n/a
Web UI (component, accessibility)	32	n/a

We checked feasibility with a pre-registered validation run on 7 October. The thresholds were fixed before anyone saw a result. Ten tasks were each run twice, once clean and once with the injected fault, using the local model llama3.2:3b. Granite:

- **detected the fault in all ten faulty runs;**

- **raised no false alarm on the clean runs;**
- **ranked the true origin first in all ten cases;**
- **confirmed it in nine;**
- **refuted the innocent step in all ten;**
- **completed all twenty runs.**

Every pre-registered threshold was met. The one finding on a clean run was a genuine arithmetic error by the model. Granite caught it although nobody had injected it.

8.2 Work Remaining

Several Iteration 1 items are late or incomplete:

- **The public demo is live, but deployment is not automatic. The Render deploy hook still has to be added as a repository secret.**
- **The main branch is not protected, and no release tag exists.**
- **Only one of the three members has opened pull requests.**
- **No member has yet given a file-level walkthrough of their module.**
- **Redaction of personal data before storage is written and tested, but not applied.**
- **A usability check with three outside users is not yet scheduled.**
- **Change record C-01 still awaits the supervisor's signature.**

Each has an owner and a date in the recovery plan. The deployment secret, branch protection and tag come before the Mid-1 meeting, as do one reviewed pull request and a walkthrough from every member. The usability check runs in Week 9, and redaction in Week 10.

Of the nineteen functional requirements set for Mid-1, fourteen are met, four are partly met and one is not:

- **A finding is opened for each failing outcome check, rather than one combined finding (deviation C-02h).**

- **The origin step is ranked first in ten of ten real-model tasks. The stricter criterion, 95% over twenty runs, has not been measured.**
- **The report export has no PDF. That is planned for FYP-1 Final.**
- **The history view has no project filter, because only one project exists.**
- **Redaction before storage (FR-SEC-01) is the requirement not met.**

8.3 Individual Responsibilities

The three ownership areas approved at the proposal defense are unchanged, and now map onto the product modules:

- **Shahoud Shahid owns Workflows, Data and Experience: the Run and Import modules, the Fault Lab and benchmark, the language-model provider layer, redaction, and the web interface.**
- **Muhammad Noor Ul Haq owns the Investigation Engine: the check language and checks engine, the trace-back attribution algorithm, and the evaluation harness with its baselines and metrics.**
- **Saif Shahzad owns Verification and Platform: replay verification and the verdict rule, the review lifecycle, reporting, the API, database and background jobs, CI/CD, deployment and security.**

The report is divided equally, with each member writing the chapters closest to their own area, about a third each:

- **Shahoud: the front matter, Chapters 1, 3, 9 and 10, and Appendices A and F.**
- **Noor: Chapters 2, 4 and 8, and Appendices B and E.**
- **Saif: Chapters 5, 6 and 7, and Appendices C and D.**

Beyond the report, Shahoud is responsible for the vision, use cases, wireframes, user manual, usability testing and poster. Noor is responsible for the literature and product review, requirements, algorithm design and evaluation. Saif is responsible for the architecture, data and behaviour models, test plan, CI/CD, deployment, and the setup and tool manuals.

Architecture decisions, integration testing, code review, ethics and defense preparation are shared. Every member is expected to demonstrate the full investigation journey alone. Table 8.2 summarises accountability.

Table 8.2: Responsibility matrix (R responsible, A accountable, C consulted, I informed)

Deliverable	Shahoud	Noor	Saif
Runner, workflows, faults, SDK	R/A	C	C
Checks, attribution, evaluation	C	R/A	C
Verification, review, API/DB, CI/CD, deployment	C	C	R/A
UI consistency, report integration	R/A	R	R
Supervisor minutes, Form 10, milestone log	R/A	I	I
Form 13 and AI usage log	R	R	R/A
Release tagging	C	C	R/A

8.4 Contribution of Each Team Member

We report contribution as the repository shows it. Excluding merges, the main branch holds 82 commits: 40 from Shahoud, 40 from Noor and 1 from Saif, who also made seven merge commits on the proposal-stage branch.

The history falls into three phases:

1. **Proposal stage, late August to the defense on 11 September. The work was shared. Shahoud led the early literature review and the proof-of-concept spikes. Noor produced most of the proposal deck and the defense preparation.**

2. **No commits between 14 September and 4 October.**

3. **6 to 11 October. The entire product was built in AI-assisted sessions run by Shahoud, and committed from his account.**

Since 11 October, Noor has deployed the product, verified it against the public URL, and opened the project's first two pull requests. One records the deployment and its evidence. The other fixes defect D-26, found by repeating the system tests on a second machine.

So the ownership described in Section 8.3 is not yet visible in the commit history for the implementation. The team recognises this as the main process weakness of Iteration 1. The recovery steps are in Section 8.2: each member makes real changes to their own module, through reviewed pull requests from their own account.

8.5 Repository and Version Control Evidence

The code is in a private GitHub repository, Shahoud867/Final-Year-Project, with main as the default branch. Commit messages follow Conventional Commits, such as `feat(core): ...` or `docs(deploy): ...`.

Continuous integration runs on every push and pull request, in four jobs:

- **Python: ruff, mypy in strict mode, pytest with a 90% coverage gate, bandit and pip-audit.**
- **Web: ESLint, the TypeScript compiler, Vitest, a production build and npm audit.**
- **Secret scan: gitleaks.**
- **Container: builds the Docker image and smoke-tests it.**

The continuous deployment workflow builds and tests the image, publishes it to the GitHub Container Registry, triggers Render, and smoke-tests the public URL. Its first run, on 11 October, published the image. The deploy step will run once the hook secret is configured.

The agreed branching strategy (ADR-010) is that nobody pushes to main. Each change goes on a short-lived branch named by type (feat, fix, docs or chore), is opened as a pull request, is checked by CI, and is reviewed by another member before a squash merge. Milestones are marked by tags such as v0.1.0-mid1 and v0.2.0-final1. Pushing a tag creates a GitHub Release with a tagged container image.

Up to 11 October, the history shows direct pushes to main. The first two pull requests were opened on 11 October. Branch protection and the first tag are scheduled before the Mid-1 meeting.

8.6 Milestones and Commit Evidence

- **11 September: the proposal was approved at the defense, with five panel comments.**
- **6 October: the master plan was revised to version 1.1, the repository was reorganised, and the core library and Refund Desk workflow were working.**
- **7 October: the API, web interface, CI definition and pre-registered feasibility run were completed.**

- **9 October: the code reached GitHub and the first CI run passed, including the container build and smoke test.**

- **11 October: the design pack, system tests, deployment workflow, coding standards and guide page were added, and the registers were re-checked. The public demo went live. Defect D-26 was found and fixed.**

Each milestone is backed by a commit, a CI or CD run, or an evidence file in the repository.

The core slice planned for Week 8 was delivered between 6 and 11 October. The late items are fully automated deployment, branch protection, the release tag, pull requests from every member, and the code walkthroughs. The recovery plan in Section 8.2 covers all of them.

8.7 Tools and Tool-Assisted Development

Granite is written in Python 3.11, with FastAPI, SQLAlchemy and Pydantic on the server, and React with TypeScript and Vite in the browser. Testing uses pytest, Hypothesis for property-based tests, Vitest, Testing Library, and axe-core for accessibility. Code quality and security are enforced with ruff, mypy, ESLint, bandit, pip-audit, npm audit and gitleaks. The system is packaged with Docker, built and deployed through GitHub Actions, and hosted on Render. The Refund Desk's language-model steps run on llama3.2:3b through Ollama.

We used generative AI tools throughout. The AI usage log and Form 13 disclose this in full.

During the proposal stage, a general-purpose AI assistant helped with literature-gap synthesis, planning documents, the eight proof-of-concept spike scripts, and defense material. The team ran every spike and checked its output against ground truth. This caught several errors, including a scoring bug and a mislabelled preprint.

Iteration 1, from 6 to 11 October, was implemented with Claude Code, an AI coding agent. This covered the core library, workflow, API, web interface, tests, CI/CD, diagrams and manuals. We class it as core-assist at its upper bound.

The team specified the design in the master plan before any code was written: the verdict rule, the attribution weights, the check-language limits, the state machine and the evaluation protocol. We validated the output by running it: the automated tests, strict type checks, security scans, a run on a real local model, a browser walkthrough of the full journey, and the pre-registered feasibility run.

The deployment verification and the D-26 fix after 11 October were also AI-assisted. Noor ran the checks, and the fix was submitted for review as a pull request.

Ollama and llama3.2:3b are part of the system under test, not part of Granite's reasoning, because the checks, attribution and verdict are deterministic code.

The team accepts that each member has still to show they can explain, modify and defend the AI-assisted code in their own module. That is scheduled before the evaluation.

8.8 External Components and Dependencies

Granite depends on a small number of permissively licensed components.

- **Distributed runtime: Pydantic, FastAPI, SQLAlchemy and React, all MIT; uvicorn, BSD-3-Clause.**
- **Development and test tools: pytest, ruff, mypy, Vite and ESLint, all MIT or Apache-2.0.**
- **MPL-2.0 tools: Hypothesis and axe-core. Both are used unmodified, for testing only, and are not shipped.**
- **Data: the Refund Desk's customers, orders and catalogue are a 75-order subset of the tau-bench retail dataset (15), published by Sierra under MIT. This synthetic data also serves as the source of truth for the checks.**
- **Model: Ollama (MIT) runs the local model. Llama 3.2 3B is covered by Meta's Llama 3.2 Community License. We do not distribute the weights, only the model's recorded answers, so the demo runs without a model.**
- **Hosting: Render's free plan, with container images in the GitHub Container Registry.**

Two licensing questions are still open: choosing a licence for Granite itself, and confirming the Llama 3.2 terms for the recorded answers.

8.9 Risks and Mitigation

The project keeps a register of thirty risks, reviewed weekly. Table 8.3 lists the most significant.

Table 8.3: Principal risks

Risk	P	I	Mitigation
A member cannot explain AI-assisted code ("sudden code volume")	H	H	Small pull requests, owner-written tests, walkthrough drills; cut code to what is understood
Uneven contribution visible in the repository	H	H	Each member owns a module and chapters, and commits through reviewed pull requests
Reviewers consider the work already done by DoVer or commercial tools	H	H	Honest comparison; our contribution is controlled replay with controls, executable checks and self-hosting; measure the difference
Language-model replays too unstable, giving inconclusive verdicts	M	H	Low temperature, structured outputs, deterministic tools; raise N; report the instability
Free hosting sleeps or changes during a demo	M	H	Wake the site beforehand; fall back to a laptop or Docker Compose
Feature creep	M	H	Frozen MoSCoW priorities; changes need a change record
Report diverges from the code	M	H	Diagrams generated from code and checked in CI
Users still find the interface unclear	M	H	Guide page, plain wording, usability pilot
Sandbox escape through check expressions	L	H	AST whitelist, size limits, attack tests, security scan
Real personal data uploaded to the demo	L	H	Synthetic data only, recorded mode, input limits

8.10 Security and Privacy Considerations

Granite's main attack surface is the check language, because checks are expressions that the server evaluates. Instead of Python's `eval`, we wrote our own evaluator on Python's syntax tree. It accepts only a whitelist of node types and a fixed list of functions. Each expression is limited to 500 characters, 150 syntax nodes and a depth of 14. We test twenty-five hostile expressions, and all are rejected. Property-based tests confirm that arbitrary input never crashes the compiler.

Other controls:

- **Every user input is typed and bounded. Unknown fields are refused, and re-**

quest bodies over 256 KB are rejected.

- **Evidence records are append-only and carry a SHA-256 hash, re-checked when read, so tampering is detectable.**
- **Attribution never has access to the injected fault's ground truth.**
- **An optional access token closes every endpoint except the health check.**
- **State-changing requests are limited to 120 per minute.**
- **Every response carries security headers.**
- **Secrets live only in GitHub and Render settings. A secret scan runs in CI, and dependencies are audited on every push.**

An automated test covers each of these controls.

On privacy, the public demo uses only synthetic data and recorded model answers. One requirement is not met: personal data in traces is stored without redaction (FR-SEC-01), because verification replays need the original input. A redactor exists and is tested, and a token-vault design that resolves this conflict is planned for Week 10.

Ethically, we inject faults only into our own workflows. Findings are advisory, and a person makes the final decision. Dataset licences are recorded, and usability participants will give written consent.

8.11 Current Limitations

The evidence so far is narrow. Granite has been shown on one workflow, one fault type and one small model. The feasibility run covered only ten tasks, so the confidence intervals are wide: the lower bound for confirmed origins is 60%. Four of the ten tasks were not blind, and we have made no baseline comparison yet.

Attribution is easier than usual in this workflow, because the origin step has its own check and so leaves direct evidence. Workflows without step checks fall back to a structural ranking, which we have not tested. The refutation test is also weak, since correcting the calculator cannot repair a wrong price.

The validation exposed one design limit. When a second fault sits further down the chain, correcting a single step cannot produce a confirmed verdict. This happened once, when the model also mis-added two prices (D-19). Fixing it needs a new version of the verdict rule.

The small model makes arithmetic slips in roughly one clean run in ten. These are real failures that Granite caught, not defects in Granite. For low-ranked hypotheses, rank and score can disagree (D-25), though the top ranks are unaffected.

Chapter 9

Remaining FYP Plan

This chapter sets out the plan for the rest of the project: the scope and features still to come, the remaining iterations, how they will be implemented, tested and deployed, the timeline to FYP-2, and the final deliverables.

9.1 Remaining Scope

Granite's full scope is eight product modules:

- **Run executes a multi-agent workflow and records its trace.**
- **Import brings in traces from other tools, through file upload, a Python SDK and framework adapters.**
- **Check evaluates executable checks and turns silent failures into findings.**
- **Trace back ranks the origin hypotheses.**
- **Verify tests a hypothesis by controlled replay.**
- **Review and Resolve covers the human decision, fix suggestions, recovered runs and a library of known fixes.**
- **Report provides exports, regression tests and history.**
- **Fault Lab and Evaluation generates labelled benchmark runs and measures the system against baselines.**

Iteration 1 delivered the core path: Run, Check, Trace back, Verify, Review and Report. What remains is the Import module, the second half of Review and Resolve, the PDF and regression exports, and all of Fault Lab and Evaluation.

Some things are outside the FYP altogether:

- **a multi-tenant SaaS product with billing;**
- **Kubernetes or message queues;**
- **applying fixes automatically to a user's production system;**
- **training custom models;**
- **benchmarking across seven frameworks;**
- **a preventive classifier;**
- **real customer data on the public demo.**

9.2 Remaining Modules and Features

We prioritise the remaining features by milestone, using MoSCoW. The Must items for FYP-1 Final are:

- **trace import using the Granite Trace v1 format;**
- **a second workflow, based on GSM8K maths problems;**
- **a Fault Lab with three fault types and clean controls;**
- **a Python SDK with a LangGraph adapter;**
- **LLM-generated fix suggestions, each verified as a fix experiment;**
- **recovered runs that apply a verified fix;**
- **export of a finding as a regression test;**
- **redaction of personal data;**
- **PDF reports;**
- **a first version of the evaluation module.**

For FYP-2 Mid, two more become Musts: the CrewAI adapter, and an LLM second-opinion hypothesis, which is a lightweight ErrorProbe reproduction. The evaluation module also reaches its full version.

Should items for FYP-2 are a known-fix library, GitHub login with reviewer identity, and retention controls. Notifications are a Could item. Support for further frameworks, a seven-framework compatibility layer and a leaderboard are left for future work.

9.3 Remaining Iterations

Three iterations remain.

Iteration 2, "breadth and inputs", runs from Week 9 to Week 15 of FYP-1 and ends with release 0.2.0 at FYP-1 Final. Its goal is that about half of the committed effort is built, with:

- **the core scenario working on real inputs from two sources, a live run and an imported trace;**
- **the comparison baseline measured;**
- **the deployment installed by someone other than its author;**
- **the FYP-2 evaluation plan written.**

Iteration 3, "integration and evaluation", covers the first eight weeks of FYP-2. It adds the CrewAI adapter, the known-fix library, authentication, the LLM second opinion, and a benchmark of at least 300 runs. It ends with release 0.5.0 at FYP-2 Mid.

Iteration 4, "hardening and user acceptance", runs to the end of FYP-2. It covers acceptance testing with users, fixes and handover, ending with release 1.0.0 at FYP-2 Final.

9.4 Implementation Plan

Iteration 2 is planned week by week, with each member working in their own area.

Table 9.1: Implementation Plan

Week	Shahoud	Noor	Saif
9	Trace import with a validation report	Check editor backend; fix D-25	Resume interrupted experiments
10	Two more fault types; personal-data redaction	Check editor interface; cycle bound in attribution	Fix suggestion and fix experiments
11	SDK and LangGraph adapter	Second workflow's checks; metamorphic tests	Recovered runs; regression-test export

Week	Shahoud	Noor	Saif
12	Fault Lab batch generation	Evaluation harness with last-agent and LLM-judge baselines	PDF reports; release pipeline; PostgreSQL testing
13	Benchmark v0 of at least 120 runs	Run evaluation v0; label 50 natural failures	Performance measurement; accessibility pass
14	Help page; SDK quickstart; UI fixes from the pilot	Evaluation screen	Install by a non-owner; rollback test

Alongside this, the team will act on the Mid-1 feedback in Week 9, interview stakeholders from Week 9, submit the poster in Week 13 after a usability pilot, and draft report version 2.0 in Week 14. Week 10 is kept light because of sessional exams. The SDK and adapter work in Week 11 carries the highest technical risk; if it falls behind, we will cut it back to the SDK alone.

All work follows the branching method in Section 8.5. Every member opens pull requests from their own account, another member reviews each one, and owners write the tests for their own code.

A weekly owner demo, where each member shows their own increment, doubles as rehearsal for the walkthrough the handbook requires. For the check, attribution and verification code, each owner must be able to:

- **explain it;**
- **propose an alternative;**
- **predict what breaks if it is removed;**
- **modify it live.**

9.5 Testing and Validation Plan

We treat testing, evaluation and acceptance as three separate questions: whether the system behaves as specified, whether it performs well against baselines, and whether real users would adopt it.

Testing continues at every level already in place:

- **Unit tests cover the core library, services and components, with coverage held above the 90% gate in CI.**
- **Integration tests need at least one success case and one failure case for every API endpoint.**
- **Component and accessibility tests cover the web interface.**
- **The system suite runs the user journeys against the deployed URL. Browser automation is planned.**
- **Regression testing runs everything on each pull request. A nightly recorded mini-benchmark fails the build if origin accuracy drops by more than five points.**
- **Performance tests check the response-time requirements.**
- **Security tests cover the check-language attack set, input fuzzing, secrets, dependency audits and redaction.**

The techniques are equivalence partitioning, boundary values at the verdict thresholds, a twenty-row decision table for the verdict rule, state-transition testing of every legal and illegal finding transition, property-based tests, metamorphic tests of attribution invariances, and negative tests where clean controls must produce no findings.

Evaluation grows through three benchmark versions:

- **version 0, at FYP-1 Final: at least 120 runs;**
- **version 1, at FYP-2 Mid: at least 300 runs;**
- **version 2, at FYP-2 Final: possibly 500 runs.**

Each version will have 20% clean controls, three fault types, two workflows and at least two runtimes. Two members will label a further 50 natural failures independently, and we will report their agreement. For the language-model attribution track we will use at least 30 cases from the Who&When benchmark.

We will compare Granite's provenance attribution against blaming the last agent, an LLM judge reading the whole trace, and, at FYP-2 Mid, a lightweight ErrorProbe reproduction. Ablations will vary the number of replays and controls, and remove the regression guard. Results will carry bootstrap 95% confidence intervals with fixed seeds. Each analysis will be frozen and pre-registered before it runs, as the Mid-1 feasibility run was.

Acceptance is tested in two stages. A pilot with three peers takes place before FYP-1 Final. Full acceptance testing in FYP-2 will involve five to eight participants: alumni developers, AI-course students and another FYP team. We declare them as proxies for real users. Sessions will be supervised, will last 45 minutes and will take place with consent. They will measure task completion, time, errors, assistance needed, the System Usability Scale, and time to first finding. Every issue raised will be recorded as fixed or deferred, with a reason.

9.6 Deployment Plan

The public demo has run on Render's free plan at `https://granite-demo.onrender.com` since 11 October. We created it from the repository's render.yaml blueprint, as a Docker service in recorded model mode, with a health check and automatic deploys switched off, so only code that has passed CI is deployed.

The continuous delivery workflow drives deployment. Once CI passes on main, it builds the container image, smoke-tests it and publishes it, triggers Render through its deploy hook, and smoke-tests the public URL again. The deploy hook was added on 13 October, and the workflow now deploys and verifies automatically.

The project uses four environments:

- **developers run Granite locally, or with Docker Compose, using Ollama or recorded answers;**
- **CI runs on GitHub's runners in recorded mode;**
- **the public demo runs on Render;**
- **as a self-hosted product, Granite runs on any machine with Docker Compose and a local Ollama model, following the setup manual.**

The demo currently stores its data in SQLite on temporary disk. We will test PostgreSQL support in Week 12, before any shared deployment. Before FYP-1 Final, someone who is not its author will install the system from the setup manual, we will test a rollback, and every release will go in the deployment record.

On each evaluation day we will wake the hosted instance in advance, with a laptop instance as a fallback. Recorded mode needs no network access to a model, so that fallback is reliable.

Figure 9.1 shows the deployment, and Figure 9.2 the live service.

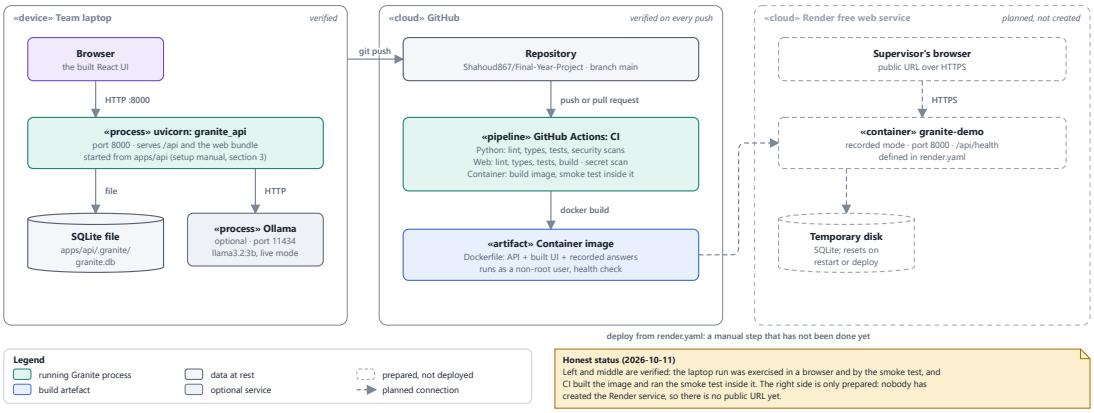


Figure 9.1: Deployment diagram: verified and planned parts

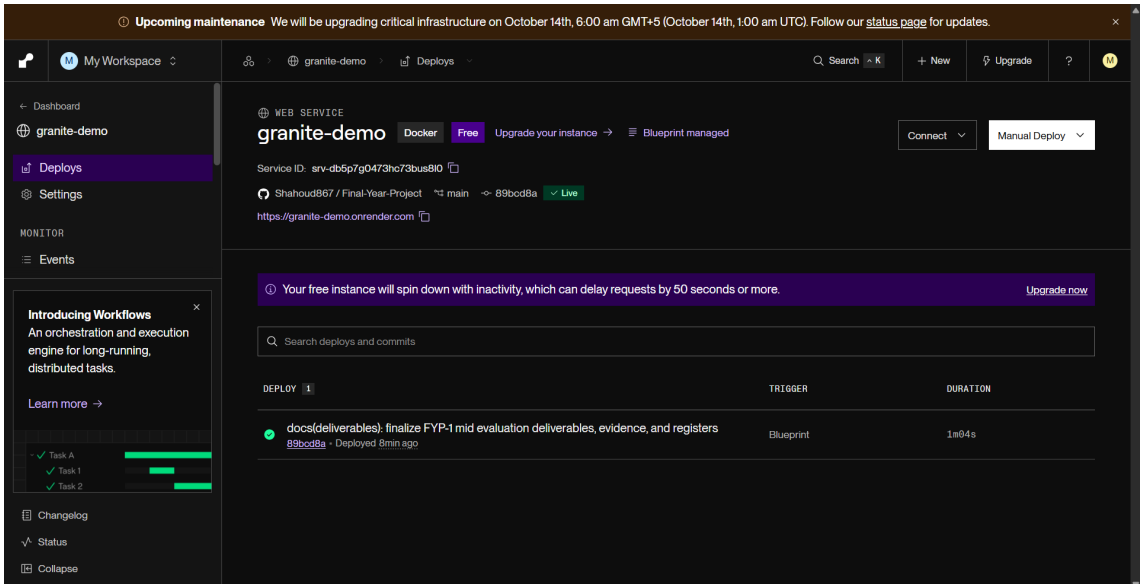


Figure 9.2: The granite-demo service live on Render

9.7 Tentative Timeline

The timeline assumes Week 1 of the semester began on 24 August.

Table 9.2: Tentative Timeline

Milestone	Week	Dates
FYP-1 Mid evaluation; tag v0.1.0-mid1	FYP-1 Week 9	12–16 Oct 2026
Import, faults, check editor, redaction, fix suggestion	Weeks 9–10	19–30 Oct
SDK and LangGraph adapter; recovered runs; poster draft	Week 11	2–6 Nov
Fault Lab, evaluation harness, PDF export	Week 12	9–13 Nov
Poster submission; benchmark v0; usability pilot	Week 13	16–20 Nov
Report v2.0; non-owner install; evaluation screen	Week 14	23–27 Nov
FYP-1 Final and poster evaluation; v0.2.0	Week 15	30 Nov – 4 Dec
CrewAI, authentication, benchmark v1, evaluation v1	FYP-2 Weeks 1–7	Spring 2027
FYP-2 Mid / Pre-Job-Fair; v0.5.0	FYP-2 Week 8	Spring 2027
User acceptance testing	FYP-2 Weeks 9–11	Spring 2027
Job Fair	FYP-2 Weeks 12–14	Spring 2027
FYP-2 Final; v1.0.0	FYP-2 Week 15	Spring 2027
Final deliverables to the committee	FYP-2 Weeks 17–18	Spring 2027

We also set internal deadlines ahead of the external ones. Code is frozen 24 hours before every evaluation. Report drafts go to the supervisor at least five days before each deadline, and the poster goes to the supervisor at least five days before Week 13.

9.8 Expected Final Deliverables

Each remaining milestone has its own set of deliverables.

At FYP-1 Final:

- **release 0.2.0, deployed;**

- **report version 2.0, covering both iterations, implementation details, the user manual, and the FYP-2 and evaluation plans;**
- **the poster;**
- **the first benchmark, and an evaluation with measured baselines;**
- **the results of the usability pilot;**
- **records of four to six stakeholder interviews;**
- **a deployment record that includes an install by a non-owner.**

At FYP-2 Mid:

- **release 0.5.0;**
- **a benchmark of at least 300 runs, with a datasheet;**
- **an evaluation report with confidence intervals and ablations;**
- **a system test report and a defect list by severity;**
- **a security scan report;**
- **the acceptance-testing plan;**
- **an updated poster and report.**

For the Job Fair we will prepare a booth demo with an offline fallback, a one-page handout with a QR code linking to the demo and the repository, and the graduate directory materials.

At FYP-2 Final the project closes with release 1.0.0 and a final report signed by the supervisor. That report covers traceability, evaluation, limitations and individual contributions. With it come the acceptance-testing report, a handover package (source code, build and run instructions, dependencies, licences, known limitations and future work), and package and deployment diagrams.

In Weeks 17 and 18, the final submission to the committee will include the report, the full presentation, references, the complete code, an executable with its README and user manual, and a passed plagiarism check.

Throughout the project, the working deployed system, its source on GitHub, its manuals, design pack and registers remain the core product deliverables.

Chapter 10

Conclusion

This report described the first iteration of Granite, a self-hostable workbench for investigating silent failures in multi-agent LLM workflows. We began from two things. The first is a gap the failure-attribution literature names itself: silent failures leave no anomaly signal. The second is the panel's feedback that our proposal described mechanisms, not a product. So we redefined the project around one workflow and one unit of value, the verified finding.

In the first iteration we built and deployed a working slice:

- **a five-agent refund workflow on public retail data;**
- **executable checks against a source of truth;**
- **deterministic provenance attribution;**
- **verification by controlled replay, with controls, under a fixed verdict rule;**
- **human review and tamper-evident evidence.**

The system has 381 automated tests and continuous integration. Its design artefacts are generated from the code. A pre-registered feasibility run met every threshold, caught a genuine model error, and exposed one design limit.

The results are deliberately narrow: one workflow, one fault type, one small model and ten tasks, with no baseline comparison or user study yet. Process gaps remain too. The main one is that per-member ownership of the code is not yet visible, and we are fixing that through reviewed pull requests and code walkthroughs.

Iteration 2 widens the inputs, workflows and fault types. It adds repair and recovery, and produces the first evaluation against baselines. By FYP-1 Final, the claims in this report can then be tested on a labelled benchmark with clean controls.

Appendix A

Proposal Action Register

Table A.1: Proposal Action Register

ID	Source	Panel comment	Owner	Due	Status	Action taken	Evidence	Remaining step
A1	Defense, 11 Sep 2026	"UI/UX needs improvement."	Shahoud (design system); all for their own screens	Mid-1	Partly	Five task-based screens, one vocabulary, a four-stage stepper. Loading, empty and error states. Light and dark themes. A keyboard-operable graph, a phone layout and plain field messages. Automated accessibility checks. A before/after comparison with the defense PoC, and a usability protocol with thresholds	`evidence/demos/` screenshots; `2026-10-13_ui-before-after.md`; accessibility audit; usability protocol	Run the usability check with three outside participants
A2	Defense	"Features are not sufficiently clear."	All	Mid-1	Done	Eight features in one fixed order, each with its purpose, inputs and outputs. The same list appears as a page in the product	Feature catalogue; guide page	Covered by this report
A3	Defense	"Unclear what inputs each feature will accept."	All	Mid-1	Done	Every input is typed and limited, and refused with a message. Trace JSON Schema and API docs published. A test keeps the guide equal to the server's limits	Data dictionary; validation tests	Inputs of features not yet built

ID	Source	Panel comment	Owner	Due	Status	Action taken	Evidence	Remaining step
A4	Defense	“Unclear how flaws will be verified and results captured.”	Saif	Mid-1	Done	Levels V0–V3. N replays plus M controls, under VR-1, with a Wilson interval. Hashed evidence, review records and a report	Verification sequence; decision-table tests; feasibility results	Larger evaluation
A5	Defense	“Overall workflow must be redefined.”	All	Mid-1	Done	One workflow, enforced by a state machine and shown as a stepper, with flow and activity diagrams	State machine; activity diagram	Supervisor’s signature on change record C-01
A6	Implicit	PoC showed research, not a product	Saif; all	Mid-1	Done	A working slice. CI on GitHub. Public deployment on Render (11 Oct). Automated CD, deploying and verifying since 13 Oct. Release tagged v0.1.0-mid1. All four pull requests reviewed and merged	Live URL; CI/CD runs; deployment record (PR #1); tag v0.1.0-mid1	Branch protection on `main`; pull requests from every member

Appendix B

Detailed Requirements

Table B.1: Detailed Requirements

ID	Requirement	Pri.	Milestone
FR-IMP-01	**Import Granite Trace v1 and Who&When JSON, up to 5 MB and 500 steps, with a validation report that gives the JSON path of every error**	**M**	**Final-1**
FR-IMP-02	**Imported runs are marked non-replayable, and get evidence-level verification only**	**M**	**Final-1**
FR-SDK-01	**A Python SDK captures a LangGraph workflow's node inputs and outputs into Granite Trace v1, through a decorator or callback**	**M**	**Final-1**
FR-SDK-02	**CrewAI adapter**	**M**	**Mid-2**
FR-CHK-05	**Create, edit, dry-run and disable checks from the UI. Edits are versioned**	**M**	**Final-1**
FR-ATT-03	**Tracing stops on cyclic lineage, within a revisit bound (default 3)**	**M**	**Final-1**
FR-ATT-04	**An optional LLM second-opinion hypothesis, labelled unverified**	**S**	**Final-1 (M at Mid-2)**
FR-VER-04	**Verification of a non-replayable run is refused, with a message**	**M**	**Final-1**
FR-VER-05	**A partial experiment keeps its completed replays, returns verdict ERROR, and can resume**	**M**	**Final-1**
FR-FIX-01	**Suggest a fix, either one atomic field value or one prompt patch, and verify it as a fix experiment**	**M**	**Final-1**
FR-FIX-02	**Promote a passing replay of a confirmed fix to a recovered run**	**M**	**Final-1**

ID	Requirement	Pri.	Milestone
FR-FIX-03	**Store verified fixes by signature, and suggest them for new findings**	**S**	**Mid-2**
FR-REP-02	**Export a regression case (JSON plus pytest) that runs in recorded mode in CI**	**M**	**Final-1**
FR-LAB-01	**Batch-generate labelled runs over workflow × runtime × fault, with a share of clean controls (default 20%)**	**M**	**Final-1**
FR-EVA-01	**Compute detection recall, false-alarm rate, top-1 and top-3 origin accuracy, verdict precision and recovery rate, with bootstrap 95% intervals, against last-agent and LLM-judge baselines**	**M**	**Final-1 v0 / Mid-2 v1**
FR-AUTH-01	**Sign in with GitHub. Reviews record the authenticated identity**	**S**	**Mid-2**
FR-ADM-01	**Delete a run and its derived records; configure retention**	**S**	**Mid-2**

Section 4.5 lists the non-functional requirements, NFR-01 to NFR-15.

Appendix C

Detailed Test Cases

Table C.1: Detailed Test Cases

ID	Test case	Requirement area	Result
SYS-CORE-01	**A full investigation of a silent failure**	**Run, detect, trace back, verify, review, report**	**Pass**
SYS-CORE-02	**A clean run raises no alarm**	**Detect**	**Pass**
SYS-CORE-03	**Refute a wrong hypothesis, then confirm the right one**	**Verify**	**Pass**
SYS-CORE-04	**The run graph marks the failing steps**	**Inspect**	**Pass**
SYS-CORE-05	**Runs and findings can be found again**	**History**	**Pass**
SYS-CORE-06	**The running system documents its own interface**	**Guide / API docs**	**Pass**
SYS-CORE-07	**The same server delivers the web interface**	**Deployment**	**Pass**
SYS-ERR-01	**Invalid input is refused, one message per field**	**Input validation**	**Pass**
SYS-ERR-02	**Asking for something that does not exist**	**Errors**	**Pass**

ID	Test case	Requirement area	Result
SYS-ERR-03	**The lifecycle cannot be skipped**	**Review, lifecycle**	**Pass (an intermittent failure was fixed as D-26)**
SYS-ERR-04	**Recorded mode cannot answer a task nobody recorded**	**Run**	**Pass**
SYS-ERR-05	**A hypothesis that cannot be tested is refused, not guessed**	**Verify**	**Pass**
SYS-ERR-06	**Limits are enforced**	**Security**	**Pass**
SYS-ERR-07	**The language model is down**	**Reliability**	**Pass**
SYS-SEC-01	**Every response carries the security headers**	**Security**	**Pass**
SYS-SEC-02	**Evidence is append-only**	**Evidence**	**Pass**
SYS-SEC-03	**Ground truth stays out of the evidence**	**Attribution integrity**	**Pass**
SYS-SEC-04	**With a token configured, only the health endpoint is public**	**Security**	**Pass**
SYS-SEC-05	**Too many state-changing requests are throttled**	**Security**	**Pass**
SYS-REL-01	**A restart loses nothing**	**Reliability**	**Pass**
SYS-REL-02	**Concurrent work is not lost**	**Reliability**	**Pass**
SYS-REL-03	**Recorded runs are reproducible**	**Determinism**	**Pass**

Appendix D

Repository and Contribution Evidence

Repository: https://github.com/Shahoud867/Final-Year-Project. **It is private, so the supervisor needs read access.**

Branching method: `.github/CONTRIBUTING.md` **and ADR-010.**

CI and CD records:

- **CI run 37924480165 (9 Oct), the first green run.**
- **CI run 38137714669 and CD run 38137856992 (11 Oct).**
- **CD run 38436983709 (13 Oct), the first fully automatic deployment, which triggered Render and passed the public smoke test.**
- **CD run 38440274746 (13 Oct), for the release tag v0.1.0-mid1.**

Deployment: `evidence/repo/2026-10-11_render-deployment.md`**.**

Note on the template. We used the prescribed LaTeX template without changing its style. We corrected four rendering defects so that it compiles as intended:

- **the class file printed the contents, figure and table lists twice;**
- **the project title and student names used on the Students' Submission page were not defined;**
- **the `longtable` package, which the template's own tables need, was not loaded;**
- **the front-matter lists and the submission table ran together on one page.**

Appendix E

Additional Design Diagrams

Table E.1: Additional Design Diagrams

Figure	Diagram	File
E.1	**Domain model / class diagram**	`domain-model.svg`
E.2	**Component diagram**	`component-diagram.svg`
E.3	**Layer diagram**	`layer-diagram.svg`
E.4	**Structure chart**	`structure-chart.svg`
E.5	**Investigation flow diagram**	`investigation-flow.svg`
E.6	**DFD level 0 (context)**	`dfd-context.svg`
E.7	**DFD level 1**	`dfd-level-1.svg`
E.8	**Activity diagram (five swimlanes)**	`activity-investigation.svg`
E.9	**Finding state machine**	`finding-state-machine.svg`
E.10	**Refund Desk data-flow graph**	`refund-desk-dataflow.svg`
E.11	**Call graph (38 functions, 49 calls)**	`call-graph.svg`
E.12	**Run sequence diagram**	`run-sequence.svg`
E.13	**Verification sequence diagram**	`verification-sequence.svg`
E.14	**Interaction overview diagram**	`interaction-overview.svg`
E.15	**ER diagram**	`database-schema.svg`
E.16	**Deployment diagram**	`deployment-diagram.svg`
E.17	**Use case diagram**	`use-case-diagram.svg`
E.18	**Algorithm diagrams (check language, attribution, VR-1, replay plan)**	`algorithm-design.md`



Figure E.1: Investigation flow diagram

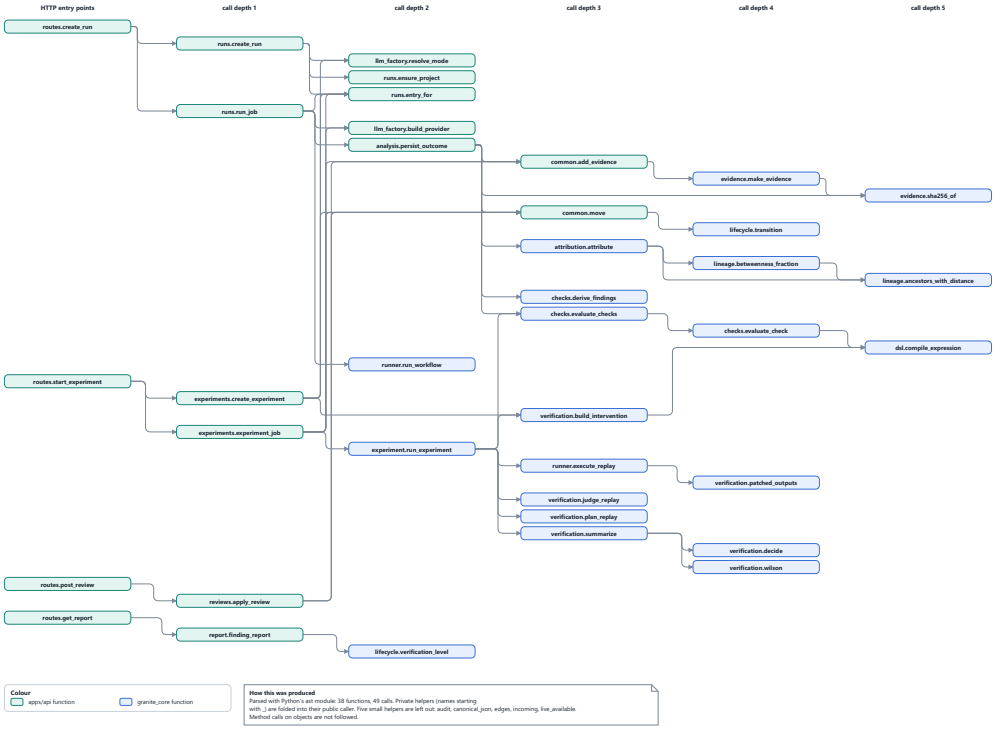


Figure E.2: Call graph computed from the source (38 functions, 49 calls)

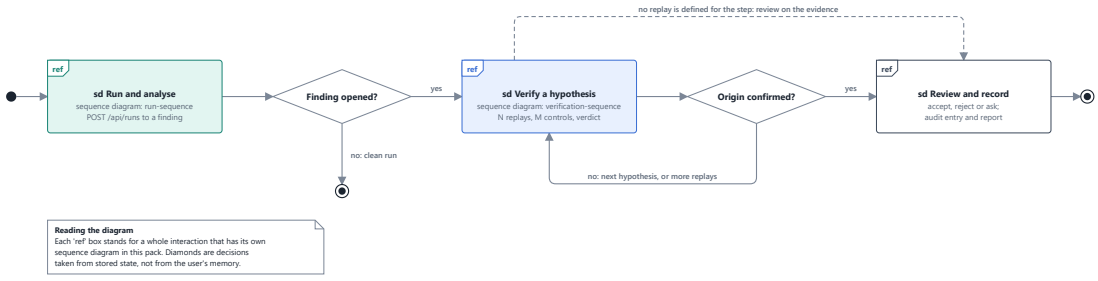


Figure E.3: Interaction overview diagram

Appendix F

Meeting and Progress Evidence

Include relevant meeting records, progress evidence, or supervisory records where required.

Table F.1: Meeting and Progress Evidence

Date	Attendees	Discussed / decided	Actions (owner, due)
11 Sep 2026	**Panel, supervisor, team**	**Proposal defense. Approved, with five comments (A1–A5)**	**Redefine the workflow; improve UI/UX; define the inputs and the verification**

The table above lists the meetings recorded in the project repository. Progress since the defense is recorded in the milestone log and the repository history. The main dated milestones are below.

Table F.2: Meeting and Progress Evidence

Date	Milestone	Evidence
6 Oct 2026	**Master plan revised to v1.1. The workflow was redefined around the verified finding, the repository was reorganised, and the core library and Refund Desk were working**	**Master plan; change record C-01; commits on `main`**
7 Oct 2026	**API, web interface and CI definition. Pre-registered feasibility run, which met all six thresholds**	**Feasibility results; demo screenshots**
9 Oct 2026	**Code on GitHub. First green CI run, including the container build and smoke test**	**CI run 37924480165**

Date	Milestone	Evidence
11 Oct 2026	**Design pack of 21 diagrams, system tests, the CD workflow and the registers. Public deployment on Render. First pull requests**	**CI/CD runs; deployment record; PR #1, PR #2**
13 Oct 2026	**Before/after UI comparison and usability-check protocol for panel action A1. Report template imported with a full first draft. All four pull requests merged. Deploy hook configured, so deployment became automatic. Release tagged v0.1.0-mid1**	**PR #3, PR #4; CD runs 38436983709 and 38440274746; tag v0.1.0-mid1**

Progress evidence: the milestone log (`docs/registers/milestone-log.md`), with planned and actual dates; the delays and recovery plan in Section 3.8; and change log entries C-01 to C-03.

Bibliography

[1] **S. Zhang, M. Yin, J. Zhang, J. Liu, Z. Han, J. Zhang, B. Li, C. Wang, H. Wang, Y. Chen, and Q. Wu, "Which agent causes task failures and when? On automated failure attribution of LLM multi-agent systems," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025, arXiv:2505.00212.**

[2] **G. Zhang, J. Wang, J. Chen, W. Zhou, K. Wang, and S. Yan, "AgenTracer: Who is inducing failure in the LLM agentic systems?" in *International Conference on Learning Representations (ICLR)*, 2026, arXiv:2509.03312.**

[3] **K. Sun, W. Li, B. Dong, Y. Lin, J. Zhang, and B. Shi, "Scope delineation before localization: A two-stage framework for enhancing failure attribution in multi-agent systems," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI-26)*, 2026.**

[4] **Anonymous, "Seeing the whole elephant: A benchmark for failure attribution in LLM-based multi-agent systems," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL), Long Papers*, 2026.**

[5] **J. Li, E. Yilmaz, B. Chen, and D.-T. Le, "Towards self-improving error diagnosis in multi-agent systems," in *Findings of the Association for Computational Linguistics: ACL 2026*, 2026, pp. 2063–2077.**

[6] **B. Lin, K. Yang, Z. Tan, Y. Lai, C. Zhang, G. Zhang, X. Yu, M. Yu, X. Wang, Y. Zhang, and Y. Wang, "AgentAsk: Multi-agent systems need to ask," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL), Volume 1: Long Papers*, 2026, pp. 28 055–28 077.**

[7] **K. Kawamura, S. Waki, and K. Tateno, "PROTEA: Offline evaluation and iterative refinement for multi-agent LLM workflows," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL), Volume 3: System Demonstrations*, 2026, pp. 27–35.**

[8] **M. Ma, J. Zhang, F. Yang, Y. Kang, Q. Lin, S. Rajmohan, and D. Zhang, "DoVer: Intervention-driven auto debugging for LLM multi-agent systems," arXiv preprint arXiv:2512.06749, 2026.**

[9] **LangChain, "LangSmith engine," Product documentation, 2026, accessed October 2026.**

[10] **Patronus AI, "Percival: agent debugging," Product documentation, 2026, accessed October 2026.**

[11] **Galileo, "Insights engine," Product documentation, 2026, accessed October 2026.**

[12] **Raindrop, "Raindrop: monitoring for AI agents," Product documentation, 2026, accessed October 2026.**

[13] **Invariant Labs, "Invariant analyzer," Open-source project documentation, 2025, accessed October 2026.**

[14] **International Engineering Alliance, "Seoul accord: graduate attributes and knowledge profile (complex computing problems)," International Engineering Alliance, accessed October 2026.**

[15] **Sierra, "τ-bench: retail domain data," https://github.com/sierra-research/tau-bench, 2024, mIT licence.**